



UNIVERSITÉ CATHOLIQUE DE LOUVAIN
ÉCOLE POLYTECHNIQUE DE LOUVAIN
COMPUTER ENGINEERING DEPARTMENT

Multi-Objective Large Neighborhood Search

Pierre SCHAUS
Promoter
Cyrille DEJEMEPPE
Reader
Florence MASSEN
Reader

Master's thesis submitted for the graduation of
Master 120 ECTS in Computer Science
option Artificial Intelligence
by Renaud HARTERT

Louvain-la-Neuve
Academic year 2012 - 2013

Aknowlegment

I would like to express my gratitude to Pierre Schaus, my promoter and former internship supervisor for his guidance, encouragements, and above all the patience he gave me during all this academic year. Especially, I would like to thank him for having pushed me to present my work at the ORBEL27 conference and for the papers that we have submitted to CP2013.

Special thanks go to Gauthier Doquire who welcomed me in his office and who read the preliminary version of this master's thesis.

I would also like to aknowledge Cyrille Dejemeppe who proposed the idea of hierarchical reasoning inside quad-trees. It is by merging our ideas that the improved deletion algorithm of Section 7.3.3 is born.

Last but not least, I would like to thank Loredana Tarallo for all the love and support she gave me during the most stressful parts of this first year living together.

Contents

Introduction	1
1 Constraint Programming	5
1.1 Constraint	5
1.2 Constraint Satisfaction Problem	7
1.2.1 Complexity of CSP	8
1.3 Constraint Propagation and Consistency	8
1.3.1 Level of Consistency	8
1.3.2 Propagation	9
1.4 Search	10
1.4.1 Exploration	10
1.4.2 Branching strategies	11
1.4.3 Discrepancy Search	13
1.5 The N-Queens Problem	13
1.5.1 Variables	14
1.5.2 Constraints	14
1.5.3 Search tree	15
1.5.4 Exploration	15
1.6 Constrained Optimization Problems	16
1.7 Large Neighborhood Search	17
1.7.1 Relaxation	18
1.7.2 Exploration	18
1.7.3 Stop Criterion	18
2 MO Combinatorial Optimization	19
2.1 Problem definition	19
2.1.1 Pareto-dominance	20
2.1.2 Pareto front	21
2.1.3 Weak and Strong dominance	21
2.2 Archive and quality	22
2.3 Non-supported solutions	25
2.4 Multi-Objective Problems	25
2.4.1 Multi-Objective Knapsack Problem	25
2.4.2 Multi-Objective Quadratic Assignment Problem	26
2.4.3 Bi-Objective Tank Allocation Problem	26

3	Related Work	27
3.1	A complete bi-objective algorithm	27
3.2	PICPA	28
3.2.1	Boxes and Box spaces	28
3.2.2	Bisection-Contraction	29
3.2.3	Instantiation and box elimination	29
3.2.4	Discussion	30
3.3	Multi-objective branch and bound	30
3.3.1	Filtering rule	31
3.3.2	Complexity	32
4	MO Large Neighborhood Search	33
4.1	The Pareto constraint	34
4.1.1	Filtering	34
4.1.2	Complexity	35
4.1.3	MO-CP versus MO-LNS	35
4.2	Optimization modes	36
4.2.1	Variable-Objective	37
4.2.2	Intensification	37
4.2.3	Diversification	38
4.2.4	Impact of diversification and intensification	39
4.3	Selection strategies	40
4.3.1	Random selection	40
4.3.2	Local Space Strategies	40
4.3.3	Nearest Neighbor Strategy	42
4.3.4	Impact of the selection strategy	43
5	Modeling Abstractions in Oscar	45
5.1	Multi-Objective Constraint Programming	45
5.1.1	Variables	46
5.1.2	Objectives	46
5.1.3	Constraints	47
5.1.4	Exploration	48
5.1.5	Starting the Search	48
5.1.6	Solutions and Archive	49
5.1.7	Example: the Multi-Objective Knapsack Problem	50
5.2	Multi-Objective Large Neighborhood Search	52
5.2.1	Solution Selection	52
5.2.2	Intensification and Diversification	53
5.2.3	Relaxation	53
5.2.4	Search	54
5.2.5	Example: the Multi-Objective Knapsack Problem	54
6	Experimental Results	57
6.1	Multi-Objective Quadratic Assignment Problem	57
6.1.1	MO-LNS settings	58
6.1.2	Results	58
6.2	Multi-Objective 0/1 Knapsack Problem	59
6.2.1	MO-LNS settings	59
6.2.2	Results	59

6.3	Bi-Objective Tank Allocation Problem	59
6.3.1	MO-LNS settings	61
6.3.2	Results	61
7	Data Structures	63
7.1	Linear-list	64
7.2	Balanced Linked-Tree	64
7.2.1	Maintaining the linked-list	65
7.2.2	Domination-free insertion	66
7.2.3	Complexity	68
7.3	Quad-trees	69
7.3.1	Domination-free Quad-Trees	70
7.3.2	Domination-free insertion	71
7.3.3	Improved deletion	72
7.3.4	Complexity	74
	Conclusion	77
	Future Work	79
	Bibliography	81

Introduction

Combinatorial optimization drives our modern society. It ensures the safe transportation of chemicals, schedules trains and planes, and allocates resources to erect skyscrapers and bridges. Combinatorial optimization organizes cancer treatments, controls the production and distribution of drugs, and provides energy to millions of people through the world. During the last decades, optimization has grown rapidly, opening up to new areas, and helping industries and companies to face complex and critical situations (as today financial crisis) and to become more and more cost-effective.

In general, combinatorial optimization problems are difficult and cannot be solved in polynomial time. Therefore, most of the work in optimization was devoted in the field of single-objective combinatorial optimization. A single-objective combinatorial optimization problem consists in finding a solution such that a given objective reaches its optimal value while respecting a set of constraints. In this situation, decisions makers have to express different objectives into a common unit (*e.g.* the cost or the benefit) and to aggregate them into a global objective to optimize.

Most part of real-world problems are multi-objective by nature. Hence, decision makers are faced with multiple – often conflicting – objectives to optimize. This raises some fundamental questions: is it truly possible to express several objectives of different natures (*e.g.* risk and cost) into a single one? In practice, people are mostly interested in a set of optimal compromises between these conflicting objectives instead of an *a priori* preference expressed by weighted aggregation.¹ This growing interest led to the emergence of Multi-Objective Combinatorial Optimization (MOCO).

Problem

Since the pioneering work of Pareto [40], many progresses were done in the field of multi-objective combinatorial optimization.² Particularly, the research community of Evolutionary Algorithm (EA) has shown a strong interest in MOCO problems. The idea behind evolutionary algorithms is to maintain and to improve a population of solutions by using evolutionary concepts as mutation and elitism. Such population-based methods seem natural for MOCO problems since we are interested in a set (*i.e.* a population) of efficient solutions.

Unfortunately, state-of-the-art evolutionary algorithms (as well as their hybridized extensions) suffer from several weaknesses. Despite the implementation

¹Section 2.3 shows that aggregation is not always able to find all the optimal compromises of a given multi-objective problem.

²See Ulungu [55] and Ehrgott [14] for reviews.

facilities offered by libraries such as ParadisEO [10] and jMetal [13], evolutionary algorithms for MOCO problems remain far from “model and run”. Users still have to provide several problem-specific and low-level implementation blocks to model their problems. This requires a great knowledge and expertise on the problems to solve and the used algorithms. Besides, evolutionary algorithms become more and more specific and strongly related to the problem to solve [15]. Therefore, evolutionary algorithms constrain modelers to spend the large part of their time to efficiently implement critical low-level blocks instead of truly modeling their problems. This lack of flexibility increases the difficulty to design single universal methods or solvers for MOCO problems.

Problem

Since the pioneering work of Pareto [40], many progresses were done in the field of multi-objective combinatorial optimization (see Ulungu [55] and Ehrgott [14] for reviews). Particularly, the research community of Evolutionary Algorithm (EA) has shown a strong interest in MOCO problems. The idea behind EA methods is to maintain and to improve a population of solutions by using evolutionary concepts as mutation and crossover. Such population-based methods seem natural for MOCO problems since we are interested in a set (*i.e.* a population) of optimal solutions.

Recently, many hybridized meta-heuristics between EA and other approaches as Local Search (LS) were proposed and obtained state of the art results on most standard MOCO problems – such as the multi-objective traveling salesman, knapsack, and quadratic assignment problems (see Abraham [1]).³ However, – despite the implementation facilities offered by libraries such as ParadisEO [10] and jMetal [13] – these approaches remain quite far from “model and run” ones. Users still have to provide several problem-specific and low-level implementation blocks – as mutations, crossover, neighborhood or moves – requiring a great knowledge and expertise on the problems and the used algorithms. Besides, meta-heuristics methods for MOCO problems become more and more specific and strongly related to the problem to solve [15]. This lack of flexibility increases the difficulty to design single universal methods or solvers for MOCO problems.

Motivation

Constraint Programming (CP) is a fundamental optimization technique that allows users to state combinatorial problems by means of constraints and let a generic solver find feasible solutions to the stated problems. The particularity of CP lies in its use of constraints to reduce the search space by removing values that cannot appear in any feasible solution. CP has shown to be competitive for solving single-objective combinatorial optimization problems in many fields [44].

Recently, the LNS (Large Neighborhood Search) framework – which combines the expressiveness of CP with the efficiency of LS – was introduced by Shaw [50]. LNS uses CP to explore neighborhood generated by removing random fragments of a best-so-far solution. LNS permitted to successfully solve

³Hybridized EA methods aims at increasing the aggressiveness of simple EA methods.

large scale problems such as vehicle routing [6, 50], scheduling [31, 39], and assignment/bin-packing problems [35, 46, 48].

Approach

We believe that the expressiveness of CP can have a real added value to tackle MOCO problems by reducing the amount of work required by the modeler. Therefore, we have dedicated this master’s thesis to extend LNS in the multi-objective context. The Multi-Objective Large Neighborhood Search framework (MO-LNS) is a population based framework which makes use of LS and CP to improve an elitist population *i.e.* a best-so-far approximation of the optimal set of solutions. The goal of MO-LNS is to quickly discover good approximations of the optimal set of solutions for large scale MOCO problems while keeping a declarative CP model.⁴

Contribution

This master’s thesis presents the multi-objective large neighborhood search framework: a new generic way of using constraint programming to solve multi-objective combinatorial optimization problems. We show that MO-LNS outperforms current state of the art multi-objective constraint programming methods on standard academic problems as well as on a complex real world problem.

Among our contributions, we introduce the **Pareto** constraint, a new global constraint to reduce the search space of MOCO problems. Besides, we propose some modeling abstractions that could be used to tackle MOCO problems with MO-LNS. We provide the reader with a basic introduction on how to use these abstractions to model MOCO problems in a clear and easy way. These modeling abstractions have been developed in the open-source OscaR solver [38] and are already part of the main distribution of the solver.

Finally, we present three data structures that could be used to store the set of optimal solutions. Particularly, we present a specific data structure to store bi-objective set of solutions as well as a new improved algorithm to store multi-objectives set of solutions.

Structure of the Thesis

The remainder of this master’s thesis is structured as follows: both first chapters provide the background necessary for the understanding of this work. Then, we present the related work on applying constraint programming to the field of multi-objective combinatorial optimization. The MO-LNS framework is then presented and detailed. The next chapter serves as an introduction to the modeling abstraction developed in OscaR. The following chapter presents our experiments on applying MO-LNS on academic problems as well as on a real-world industrial problem. The next-to-last chapter presents different data structures that could be used to store the sets of efficient solutions computed by MO-LNS.

⁴Notice that the lack of hybridization with CP methods for solving MOCO problems was recently underlined by Ehrgott in [15].

Finally, we conclude by summarizing the main contributions of this master's thesis and we present some paths of future work.

Chapter 1

Constraint Programming

Constraint Programming represents one of the closest approaches computer science has yet made to the Holy Grail of programming: the user states the problem, the computer solves it. [17]

Constraint Programming (CP) is a paradigm to solve constrained satisfaction and optimization problems. The basic idea of constraint programming is to provide a high-level declarative language that allows users to declare their problems by means of constraints. These problems are then automatically solved by generic algorithms embedded into a constraint solver.

This first chapter presents the fundamental concepts and essential definitions for the understanding of the next chapters. The reader that feels comfortable with constraint programming and large neighborhood search should be able to jump directly to the next chapter.

1.1 Constraint

A *constraint* is a logical relation defined over a set of *variables*. A constraint restricts the values that may be assigned simultaneously to its variables and is used to state properties that solutions of a problem have to hold. Variables and constraints are formally defined as follows.¹

Definition 1.1 (Variable). A variable is an abstract object that can take different values. The set of all the possible values of a variable x is called its *domain* and is denoted $D(x)$. In this master's thesis, we assume that a domain is always a finite set of discrete values.

Definition 1.2 (Constraint). A constraint $c(x_1, \dots, x_k)$ is a logical relation defined over the variables $x_1, \dots, x_k$. The value k is called the *arity* of the constraint c and the set $vars(c) = \{x_1, \dots, x_k\}$ is called the *scope* of the constraint.

A constraint can be defined by enumerating all the tuples of values that satisfy the constraint *i.e.* the logical relation. Conversely, a constraint can be

¹The following definitions are adapted from [12] and [44].

defined as an enumeration of all the tuples of values that do not satisfy the constraint. However, it is more common to define a constraint by using mathematical properties as shown in Example 1.1.

Example 1.1. Consider the variables x and y with domains $D(x) = D(y) = \{1, 2, 3, 4\}$. The constraint $x < y - 1$ restricts the domains of x and y to $D(x) = \{1, 2\}$ and $D(y) = \{3, 4\}$.

The constraint $x < y - 1$ is a *binary constraint* i.e. it is defined over two variables. Binary constraint are common in constraint programming languages and can be used to declare many problems.

One particularity of constraint programming is to offer *global constraints* that could express more complex properties than simple binary constraints. For example, the *allDifferent* constraint enforces a set of variables to be assigned to different values [42].² Notice that a global constraint can always be decomposed into a set of binary constraints. Example 1.2 presents the *binary decomposition* of a trinary *allDifferent* constraint.

Example 1.2. The constraint *allDifferent*(x, y, z) can be stated by three inequalities: $x \neq y$, $x \neq z$, and $y \neq z$. This set of inequalities is called the *binary decomposition* of the *allDifferent* constraint.

Therefore, one main interest of global constraints relies on their expressiveness. Besides, a global constraint can also exploit its larger scope to infer more information about the satisfiable combinations of values of its variables. For example, consider the *allDifferent* constraint over the variables x , y , and z with domains $D(x) = D(y) = D(z) = \{1, 2\}$. Clearly, we can infer that no combinations of values satisfy the *allDifferent* constraint. However, it is impossible to infer this inconsistency by locally considering each inequalities from the binary decomposition of the *allDifferent* constraint (see Figure 1.1). Global constraints thus have three major advantages [8, 41]:

- **Expressiveness:** a global constraint extends the expressiveness of a CP language. It simplifies the modeling task by offering a new constraint or suitable abstractions for a pattern of constraints.
- **Quality of filtering:** a global constraint can also exploit its larger scope to infer more information about the satisfiable combinations of values of its variables.
- **Computational efficiency:** for some global constraints, the filtering is the same as for its binary decomposition. However, this filtering can be obtained with a better time or space complexity. Filtering is discussed in details in Section 1.3

The computational efficiency of a constraint is determined by its embedded filtering algorithms. Global constraints encapsulate algorithms from many domains as graph theory or operational research. For example, the *allDifferent* constraint relies on a maximum matching algorithm [42]. The interested reader can refer to [44] for an introduction to the mechanisms embedded in global constraints. A catalogue of the more common global constraints is presented in [5].

²The interested reader can refer to [8] and [41] for more information on the classification of constraints and global constraints.

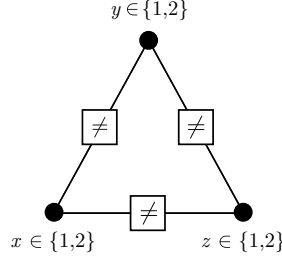


Figure 1.1: The constraint graph of the binary decomposition of the *allDifferent* constraint over variables x , y and z with domains $D(x) = D(y) = D(z) = \{1, 2\}$. Each value of x has a support value in y w.r.t. the constraint $x \neq y$ *i.e.* when assigning x to 1 (resp. 2) it is possible to assign y to 2 (resp. 1). Since this property is also true for inequalities $y \neq z$ and $z \neq x$, it is impossible to detect the unsatisfiability of this problem.

1.2 Constraint Satisfaction Problem

Now that we have a good understanding of what a constraint is, we can focus on the kind of problems that we want to solve.

As mentioned above, the central idea of constraint programming is to allow the user to declare a problem by a set of constraints. This kind of problem is called *Constraint Satisfaction Problem* (CSP) and is formally defined as follows.

Definition 1.3 (CSP). A CSP is defined by a triple $\mathcal{P} = \langle X, D, C \rangle$ composed of:

- a set $X = \{x_1, \dots, x_n\}$ of n variables;
- a domain $D(X) = D(x_1) \times \dots \times D(x_n)$ which is the cartesian product of the domains of each variable $x_i \in X$;³
- a set $C = \{c_1, \dots, c_t\}$ of t constraints defined over the variables $x_i \in X$.

To solve a given CSP, it is necessary to assign values to variables such that all constraints are satisfied. The definitions of *assignment* and *solution* are as follows.

Definition 1.4 (Assignment). An assignment A on a CSP $\mathcal{P} = \langle X, D, C \rangle$ is defined as a set of pairs variable/value $\{(x_1, v_1), \dots, (x_k, v_k)\}$ that maps each variable $x_i \in \{x_1, \dots, x_k\} \subseteq X$ to a value v_i of its corresponding domain $D(x_i)$. An assignment is complete if it assigns all the variables of the problem *i.e.* $\{x_1, \dots, x_k\} = X$; It is partial otherwise.

Definition 1.5 (Solution). A solution is a complete assignment that satisfies all the constraint $c \in C$.

Example 1.3. Let us consider the CSP $\mathcal{P} = \langle X, D, C \rangle$ such that:

$$\begin{aligned} X &= \{x_1, x_2, x_3\} \\ D(x_i) &= \{0, 1, 2, 3\}, \forall x_i \in X \\ C &= \{x_1 + x_2 + x_3 \leq 5, x_1 + x_2 + x_3 > 0\} \end{aligned}$$

³As mentioned in Definition 1.1, we assume that a domain is always a finite set of discrete values.

Therefore, $\{(x_1, 3), (x_3, 3)\}$ is a partial and inconsistent assignment with regard to the set of constraint C . However, $\{(x_1, 1), (x_2, 2), (x_3, 0)\}$ is a complete and satisfiable assignment *i.e.* a solution.

1.2.1 Complexity of CSP

The theoretical complexity of CSPs depends on the types of domains and constraints used. In some cases, the problem may be polynomial⁴. In general CSPs over finite domains – such as the ones considered in this master’s thesis – are NP-complete.

A simple way to solve a CSP may be to enumerate all the possible complete assignments until finding a satisfiable solution or proving the inconsistency of the problem if no satisfiable solution exists. Unfortunately, the search space of a CSP grows exponentially with the number of variables and the size of their domains (see Example 1.4). Hence, it is impossible to explore the whole search space of interesting problems within a reasonable computational time.

Example 1.4. Consider a CSP $\mathcal{P} = \langle X, D, C \rangle$ with n variables such that the domain of each variable contains d values. The search space of the problem $D(X) = D(x_1) \times \dots \times D(x_n)$ contains d^n different complete assignments.

1.3 Constraint Propagation and Consistency

Constraint propagation aims to use inference to reduce (or strengthen) the domains of non-assigned variables by removing inconsistent values *i.e.* values that do not belong to any solution. Therefore, constraint propagation (or simply propagation) is used to reduce the search space of a given CSP without removing any feasible solution.

Definition 1.6 (Propagation). Let $\text{sols}(\mathcal{P})$ denote all the feasible solutions of a CSP $\mathcal{P} = \langle X, D, C \rangle$. Propagating $\mathcal{P}$ produces a new CSP $\mathcal{P}' = \langle X, D', C \rangle$ such that $D'(X) \subseteq D(X)$ and $\text{sols}(\mathcal{P}) = \text{sols}(\mathcal{P}')$.

Clearly, propagation has reached a solution when the domain of each variable is strengthened to a singleton *i.e.* all variables are assigned. Conversely, the unfeasibility of a CSP is detected when propagation lead to a variable with an empty domain *i.e.* all the values of this variable are inconsistent regarding the partial assignment already built.

Example 1.5. Consider the constraint $x = y + 3$ over variable x and y with domains $D(x) = D(y) = \{1, 2, 3, 4, 5, 6\}$. Propagation reduces the domain of both variables to $D(x) = \{4, 5, 6\}$ and $D(y) = \{1, 2, 3\}$.

1.3.1 Level of Consistency

Filtering all the inconsistent values of a problem is called enforcing its *global domain consistency* [12].

⁴This is the case for linear problems stated by means of linear constraints on continuous domains.

Definition 1.7 (Global Domain Consistency). Let $D(X)_{x=v}$ denote the subspace of the search space $D(X)$ such that x is assigned to v . A problem $\mathcal{P} = \langle X, D, C \rangle$ is *global domain consistent* if all the values of its variables participate in a solution:

$$\forall x \in X, \forall v \in D(x), \exists a \in D(X)_{x=v} : \{(x_1, a_1), \dots, (x_n, a_n)\} \in \text{sols}(\mathcal{P})$$

Unfortunately, enforcing global domain consistency is an NP-Hard problem [12]. Propagation thus has to rely on weaker levels of consistency. The idea is to consider every constraint locally instead of considering the whole set of constraints at once. Each constraint is responsible of enforcing its own level of consistency through embedded filtering algorithms. Typically, designing a filtering algorithm requires a trade-off between speed and the level of consistency. The most popular of these levels are the *domain-consistency* [34] and the *bound-consistency* [57].

Definition 1.8 (Domain-Consistency). Given a CSP $\mathcal{P} = \langle X, D, C \rangle$, a constraint $c \in C$ is *domain-consistent* if and only if:

$$\forall x \in \text{vars}(c), \forall v \in D(x), \exists a \in D(X)_{x=v} : c(a_1, \dots, a_k)$$

Example 1.6. Consider the constraint $x + y = 6$ on the variables x and y with domains $D(x) = \{1, 2, 3, 4, 5\}$ and $D(y) = \{1, 5\}$. A domain-consistent filtering algorithm detects that the values 2, 3, and 4 of $D(x)$ do not have a support in $D(y)$. Therefore, they are removed from the domain of x leading to $D(x) = \{1, 5\}$ and $D(y) = \{1, 5\}$.

Definition 1.9 (Interval Domain). Given a CSP $\mathcal{P} = \langle X, D, C \rangle$, we define $[D(X)]$ as the cartesian product of the *interval domains* $[D(x_1)] \times \dots \times [D(x_n)]$ where $[D(x_i)] = [\min(D(x_i)), \max(D(x_i))]$

Definition 1.10 (Bound-Consistency). Given a CSP $\mathcal{P} = \langle X, D, C \rangle$, a constraint $c \in C$ is *bound-consistent* if and only if:

$$\forall x \in \text{vars}(c), \forall v \in \{\min(D(x)), \max(D(x))\}, \exists a \in [D(X)]_{x=v} : c(a_1, \dots, a_k)$$

Example 1.7. Consider the constraint $x + y = 6$ on the variables x and y with domains $D(x) = \{1, 2, 3, 4, 5\}$ and $D(y) = \{1, 5\}$. A bound-consistent filtering algorithm is not able to detect inconsistent values since the bounds 1 and 5 of $D(x)$ have supports in the interval domain $[D(y)]$ of y .

As shown in Example 1.6 and 1.7, bound-consistency is a relaxation of domain-consistency. However, this filtering can be achieved much faster for some constraints. For instance, enforcing the domain-consistency of a binary sum has a time complexity of $\mathcal{O}(|D(x)| * |D(y)|)$ while enforcing bound-consistency is performed in constant time $\mathcal{O}(1)$.

1.3.2 Propagation

The aim of propagation is to ensure communications between the local filtering algorithm of each constraint. In the propagation process, each constraint is called successively to strengthen the domain of its variables until reaching its fix point. The fix point is a state in which a constraint cannot perform any

additional inference.⁵ When reaching its fix point, a constraint falls asleep until the domain of one of its variables is strengthened again. Then, the constraint awakes, strengthening the domain of its variables until reaching a new – stronger – fix point. Since constraints are likely to share variables of their respective scopes, the propagation process may lead to a circular process of inference until a mutual fix point is found, meaning that no further filtering is possible.⁶

Example 1.8. Consider the binary decomposition of the *allDifferent* constraint from Example 1.2 with the partial assignment $A = \{(x, 1)\}$. Propagation over the constraint $x \neq y$ removes the value 1 from the domain of y which is now equal to $\{2\}$. Since the domain of y has been strengthened, the constraint $y \neq z$ is called and removes the variable 2 from the domain of z which is now equal to $\{1\}$. The domain of z being strengthened, the constraint $z \neq x$ is called to remove the value 1 from the domain of x resulting into an empty domain *i.e.* the partial assignment A is inconsistent.

1.4 Search

Propagation alone is hardly sufficient to find a solution to a given CSP. To address this problem, common CP solvers rely on a *divide and conquer* approach where the problem to be solved is recursively split into smaller subproblems by assigning values of one variable at a time. This process implicitly develops a search tree where each node is a subproblem of its father.

The size of the search tree is exponential and cannot be explored in a reasonable time (see Section 1.2.1). Constraint programming use propagation to reduce the size of the search tree by pruning branches that cannot lead to solutions. The search tree is thus defined as follows:

- The root is the original CSP;
- An internal node is a subproblem of its father;
- A leaf is either a *solution node* (*i.e.* all variables are assigned) or a *failed node* that contains at least one empty domain.

1.4.1 Exploration

The search tree is generally explored using a simple *depth-first-search backtracking algorithm* common in the field of artificial intelligence [21]. Each time the search algorithm expands a new node, additional information is provided to the propagation process. Propagation is called at each node to remove inconsistent values in the domain of the variables. When a failed node occurs (meaning that at least one variable has an empty domain) the search algorithm reconsiders its last choice and backtracks to another node. Figure 1.2 shows the order in which nodes are explored using the depth-first-search backtracking algorithm.

⁵An *idempotent* constraint reaches its fix point in one execution of its filtering algorithm.

⁶The first apparition of a propagation algorithm is attributed to Waltz [58]. Nowadays, different propagation algorithms have been proposed and are usually implemented in current CP solvers. For instance, OscaR implements AC3 and AC5 (see [44] for more details).

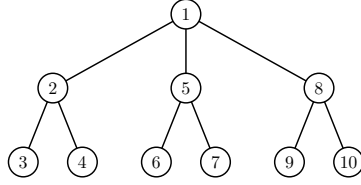


Figure 1.2: Order in which nodes are explored using a left to right depth-first-search algorithm.

1.4.2 Branching strategies

Branching is the process of defining k CSPs $\mathcal{P}_i$ from a given CSP $\mathcal{P}$ such that the resulting CSPs do not share solutions and cover all the solutions of the initial CSP [12]:

$$\begin{aligned} \text{sol}(\mathcal{P}_1) \cap \dots \cap \text{sol}(\mathcal{P}_k) &= \emptyset \\ \text{sol}(\mathcal{P}_1) \cup \dots \cup \text{sol}(\mathcal{P}_k) &= \text{sol}(\mathcal{P}) \end{aligned}$$

The branching strategy fully determines the topology of the search tree. In general, branching strategies consist in selecting a variable and then consider the values in its domain to define the CSPs of the child nodes. These branching strategies are known as *variable/value strategies*.

Variable/Value labeling. Let x be the chosen variable, the *labeling strategy* generates a CSP for each value a in $D(x)$ by posting the additional constraint $x = a$ to form the child subproblem. Hence, the labeling strategy generates $|D(x)|$ new subproblems to solve. Figure 1.3 illustrates this strategy. Notice that the order in which variables and values are selected may have a great impact on the search process.

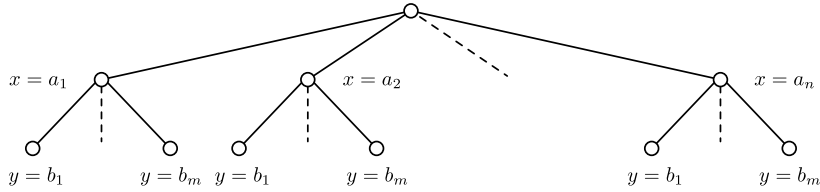


Figure 1.3: A tree developed using a labeling strategy.

Variable/Value binary labeling. In certain cases, the labeling strategy may induce a too large branching factor in the search tree. The *binary labeling strategy* could be used to impose a branching factor of 2.⁷ Let x be the chosen variable, the binary labeling strategy generates two CSPs by posting the additional constraints $x = a$ and $x \neq a$ with a a value in $D(x)$. This labeling strategy is illustrated by Figure 1.4. Notice that it is possible to consider another variable after posting the constraint $x \neq a_i$ (right branch).

⁷Binary labeling is the standard branching strategy of OcaR.

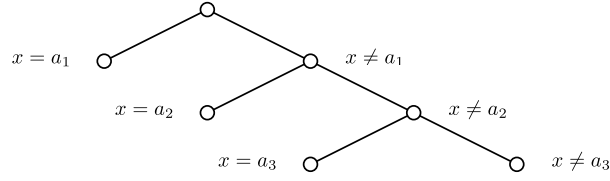


Figure 1.4: A tree developed using a binary labeling strategy.

Variable/Value domain splitting. For some problems, the decisions taken by labeling strategies may be too strong [12]. Hence, a weaker branching strategy is preferable. The *domain splitting strategy* splits the domain of a variable into two distinct parts. In general, the domain of a chosen variable x is split by posting the additional constraint $x < a$ (left branch) and $x \geq a$ (right branch) with a a value in $D(x)$.⁸ Figure 1.5 illustrates the domain splitting branching strategy.

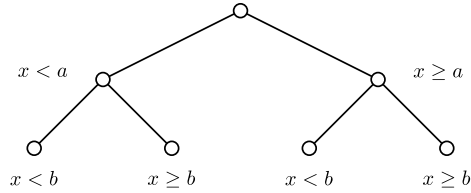


Figure 1.5: A tree developed using a domain splitting strategy.

Choosing the right variable and value is a complex problem and may have a great impact on the search process. To address this problem, branching strategies rely on heuristics to order the variables and the values to choose [44]. For instance, Figure 1.6 illustrates the influence of choosing the variables in different orders.

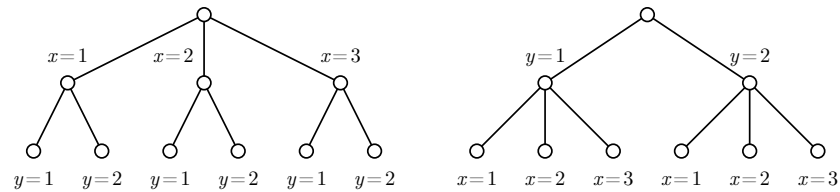


Figure 1.6: The topology and the size of a search tree are determined by its branching strategy. In this example, choosing x (resp. y) as first variable lead to a search tree with 10 (resp. 9) nodes.

Good heuristics lead the search to a solution (or to show that the problem is unsatisfiable) in the fewest number of nodes. In general, the choice of the right heuristic is problem specific. Still, general guidelines have been proposed to help

⁸As a rule of thumb, the left branch should always be generated with $x < a$ instead of $x \leq a$. Indeed $x \leq a$ may cycle if $a = \max(D(x))$.

designing good heuristics [7, 9, 21, 26]. They are presented in the remainder of this section.

Minimal tree principle. The idea behind *minimal tree principle* [21, 26] is to make choices that are likely to reduce the size of the search tree. Clearly, reducing the size of the search tree decreases the time spent exploring it to find a solution, or to prove the unsatisfiability of the CSP.

First-Fail principle. In general, the unsatisfiability of a CSP can be detected without having to instantiate all its variables. The *first-fail principle* [26] is based on this observation, and aims at detecting the unsatisfiability of a CSP as soon as possible, by considering first the variables that belong to the most critical parts of the problem.

Best-First principle. The *best-first principle* is a value ordering heuristic that aims at assigning first the value that is most likely to be part of a solution. Clearly, choosing values that do not belong to a solution does not make much sense and will lead, sooner or later, to a failed node.

Decreasing influence principle. One main issue of the standard depth-first-stage is that it is likely to remain stuck in the bottom left part of the search tree. Hence, bad choices made at the first level of the tree may have a huge impact on the search process. The *decreasing influence principle* [7, 9] recommends to spent more time to choose carefully the right variables and values at the first levels of the search tree.

1.4.3 Discrepancy Search

Discrepancy search [27, 28] is an alternative to the standard depth-first-search used to explore the search tree. A *discrepancy* occurs when the search does not take the leftmost branch of a node, that is when the search does not follow the value ordering heuristic. The idea behind discrepancy search is to iteratively explore the search tree by increasing the number of allowed discrepancies on paths from the root to a leaf node of the tree.⁹ Hence, discrepancy search favors the value ordering heuristic. Figure 1.7 illustrates the exploration of a search tree with discrepancy search.

According to the decreasing influence principle, discrepancy search can recover from mistakes made in the first levels of the search tree. Hence, it avoids remaining stuck in the bottom left part of the search tree, and brings diversification to the exploration process. The major drawback of discrepancy search lies in its space complexity that grows exponentially with the number of variables in the worst case.¹⁰

1.5 The N-Queens Problem

To illustrate the previous concepts of constraint programming, we introduce a well known “hello world” constraint satisfaction problem.

⁹As a tie breaker, discrepancies that occur near the root of the tree are preferred.

¹⁰Depth-first-searches have a linear space complexity in the number of variables.

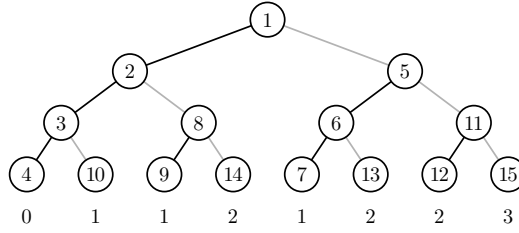


Figure 1.7: Exploration of an arbitrary search tree with discrepancy search. The order in which nodes are explored is indicated inside the nodes while the level of discrepancy of a path is indicated under its leaf.

The N-Queens Problem consists in placing n chess queens on an $n \times n$ chessboard so that no queens may be attacked by any other queen in one move. Thus, a solution requires that no pairs of queens share the same row, column, or diagonal.

1.5.1 Variables

To model this problem as a CSP, we choose to associate a variable with each queen.¹¹ Since we know in advance that there is exactly one queen in each column, the problem is to determine on which row we should place the queen of column i . We denote the variable associated with the queen of column i as r_i . The value of this variable gives the row in which the queen should be placed. We also introduce the auxiliary variables rd_i and fd_i , respectively associated with the rising and the falling diagonals of the queens. Those variables will allow us to state naturally that queens are not allowed to be placed on a same diagonal. Thus, for each queen $i \in \{1, \dots, n\}$, we define the domain of the variables as follows:

$$D_{r_i} = \{1, \dots, n\}$$

$$D_{rd_i} = \{2, \dots, 2n\}$$

$$D_{fd_i} = \{1 - n, \dots, n - 1\}$$

1.5.2 Constraints

The set of constraints contains two distinct subsets. The first one, maintains the relation between rows and diagonals:

$$\forall i \in \{1, \dots, n\} : rd_i = r_i + i$$

$$\forall i \in \{1, \dots, n\} : fd_i = r_i - i$$

The second subset adds the global constraints that prevent queens to be in a same row or in a same diagonal:

1. queens must be in different rows:

$$allDifferent(\{r_i \mid i \in \{1, \dots, n\}\})$$

¹¹This model is adapted from [51].

- queens must be in different rising diagonals:

$$allDifferent(\{rd_i \mid i \in \{1, \dots, n\}\})$$

- queens must be in different falling diagonals:

$$allDifferent(\{fd_i \mid i \in \{1, \dots, n\}\})$$

1.5.3 Search tree

The search tree of this problem is determined by a simple labeling branching strategy. It consists in selecting variables (columns) and values (rows) by increasing ordering. For instance, the first assignation is: $r_1 = 1$.

1.5.4 Exploration

In order to illustrate this example, we will consider the particular 4-Queens instance which consists in placing 4 queens on a 4×4 chessboard.

First, we consider the subtree resulting from assigning r_1 to 1 (which is the first choice of our exploration). Calling the propagation mechanism on this new CSP removes the values 1 and 2 from the domain of r_2 (see Figure 1.8a). Since the current CSP is not a solution nor a failed node, we continue our exploration by assigning r_2 to 3. Unfortunately, propagating this new CSP removes the remaining values of r_3 and leads to a failed node (Figure 1.8b). Then, the exploration reconsiders its previous choice by reassigning r_2 to 4. Propagation on this new CSP removes the value 4 of r_3 constraining it to be assigned to the value 2 (Figure 1.8c). Again, calling the propagation on this new CSP removes the remaining values of r_4 and leads to another failed node (Figure 1.8d). Finally, having considered all the leaves of this subtree, we can state that $r_1 = 1$ does not belong to a solution.

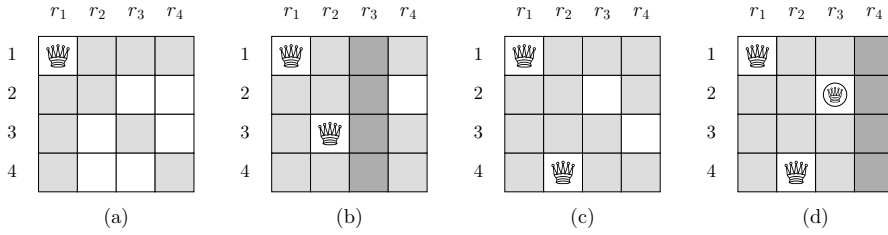


Figure 1.8: CSPs explored after assigning the variable r_1 to 1 (circled queens are inferred from propagation).

Our first choice being inconsistent, the search algorithm reconsiders it by backtracking and by assigning r_1 to 2 (see Figure 1.9a). Calling the propagation process on this new CSP removes the values 1, 2 and 3 from the domain of r_2 constraining it to be assigned to 4 (see Figure 1.9b). Since propagation as not yet reached its fix point, this algorithm is recalled and removes the value 3 of r_3 and assigns it to 1 (see Figure 1.9c). Again, propagation is recalled and removes the value 3 from the domain of r_4 (see Figure 1.9d). Finally, the mutual fix point is reached. The final CSP has been reduced into a solution without having to consider any other choice than $r_1 = 2$.

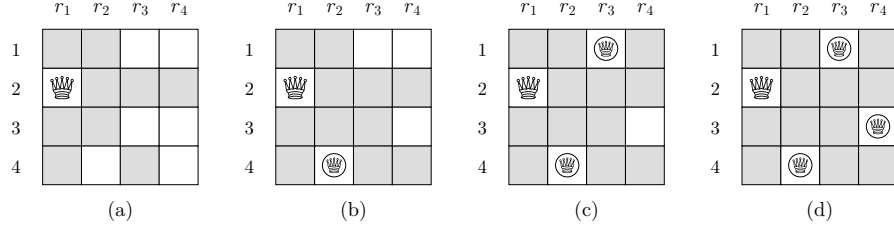


Figure 1.9: Different steps of the propagation algorithm after assigning the variable r_1 to 2.

During this search for a solution, we have explored 5 different CSPs. The actual search tree is illustrated in Figure 1.10. Observe that propagation alone is sufficient to reduce the right branch of the root into a solution.

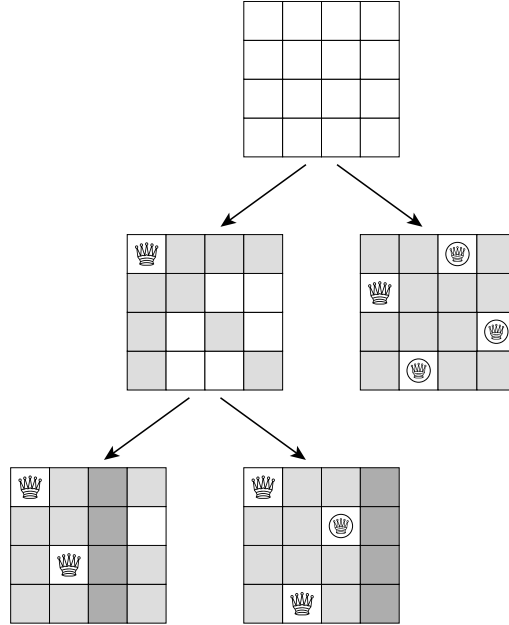


Figure 1.10: Exploration using a left to right depth-first-search strategy on a search tree generated by labeling branching strategy.

1.6 Constrained Optimization Problems

When solving a constraint satisfaction problem, one is usually interested to find one solution that satisfies all the constraints or to prove the unsatisfiability of the problem if no solutions exists. In general, there are many solutions to a given CSP. Some solutions may be preferred to others by means of an *objective function* to minimize.¹² These new problems are known as Constrained

¹²Without any loss of generality, we decided to focus on constrained minimization problems.

Optimization Problems (COP). Clearly, solving a COP is more complex than solving a simple CSP since we are interested in finding a particular solution instead of any satisfiable solution.

More formally, a COP is defined in the same way as a CSP $\mathcal{P} = \langle X, D, C \rangle$ with an objective function $f : X \rightarrow \mathbb{R}$. The goal is to find a solution of the CSP which minimizes f .

In general, the objective is included as a part of the CSP by adding a variable *obj* into X such that $X = \{x_1, \dots, x_n, obj\}$. The value of the objective variable is maintained using an *objective constraint*: $obj = f(X)$. The optimal solution of a COP is found using an iterative process:

1. Find a satisfiable solution to the problem with v the value assigned to the objective variable;
2. Adds a *tightening constraint* enforcing the objective variable to be strictly less than v .

This iterative process is repeated until it becomes impossible to find a satisfiable solution to the tightened CSP, meaning that we have reached the optimal value for the objective function f .

1.7 Large Neighborhood Search

Large Neighborhood Search (LNS) [50] is a powerful technique that hybridizes Local Search and Constraint Programming to solve difficult constrained optimization problems as scheduling problems [31, 39], routing problems [6, 50], and assignment problems [35, 46, 48]. The idea of LNS is to iteratively improve a best-so-far solution by exploring its neighborhood (*i.e.* a subspace of the search space induced by a solution) using constraint programming. The LNS process can be summarized as follows:

1. Find a first initial solution and consider it as the best-so-far solution.
2. Repeat until a stop criterion is met:
 - 2.1. Generate a new neighborhood by relaxing the best-so-far solution.
 - 2.2. Explore the generated neighborhood using CP.
 - 2.3. If a better solution is found, make it the best-so-far solution.

As shown, LNS trades the completeness of constraint programming for the efficiency of local search. Particularly, LNS addresses the scalability issue of CP (see Section 1.4.2) by exploring several regions of the search space *i.e.* the neighborhoods. Besides, one main advantages of LNS lies in its ease of implementation. Indeed, users only have to provide two main implementation blocks to turn a CP model into an LNS procedure: a relaxation procedure and an exploration limit.

1.7.1 Relaxation

Relaxation aims at generating neighborhood from a best-so-far solution by removing some parts of its structure.¹³ In general, relaxation is presented in a dual way by selecting the parts to maintain instead of selecting the parts to remove. These parts can correspond to assigned values of the solution or to more complex relations between the decision variables [32]. Usually, the selected parts are maintained by posting additional temporary constraints to the problem. These constraints restrict the search space of the problem, leading to the neighborhood to explore.

1.7.2 Exploration

The exploration phase consists in using constraint programming to explore the neighborhood induced by the relaxation procedure. As mentioned above, a neighborhood corresponds to the search space of a more constrained version of the original problem. Therefore, the size of a neighborhood can be very large (see Section 1.2.1). It is thus often necessary to fix some limits on the exploration (*e.g.* a time limit or a limit on the number of allowed backtracks) to avoid exploring a given neighborhood for too long.¹⁴ As well, a neighborhood is not guaranteed to contain better solutions than the best-so-far one.

1.7.3 Stop Criterion

Since LNS cannot provide users with proofs of optimality, it is necessary to fix a stopping criterion to the LNS iterative process. The following simple stopping criteria are commonly used:

- Limit the time spent solving the problem;
- Limit the number of LNS iterations.

In certain cases however, it may be useful to define more sophisticated stopping criteria *e.g.* a measure of stagnation.

¹³Notice that “relaxation” refers to relaxation of the solution, not to relaxation of the problem. It is also called “fragment selection” or “destroy function”.

¹⁴As a rule of thumbs, a complete neighborhood exploration should occur once every two iterations.

Chapter 2

Multi-Objective Combinatorial Optimization

This chapter introduces the second part of the background of this master's thesis. It presents multi-objective combinatorial optimizations and the kind of problems we are trying to solve. Section 2.1 gives a short overview of the definitions. Section 2.2 describes what are the solutions we are trying to find and how to evaluate them. Then, Section 2.3 describes the difficulty of multi-objective and why they cannot be tackled with standard single-objective approaches. Finally, Section 2.4 presents standard academic multi-objective problems and a complex large scale industrial problem.

2.1 Problem definition

The typical Multi-Objective Combinatorial Optimization (MOCO) problem we want to solve has m conflicting integer objectives to minimize while satisfying some constraints:

$$\begin{array}{ll} \text{Minimize} & obj = (obj_1, obj_2, \dots, obj_m) \\ \text{Subject to} & constraints \end{array} \quad (2.1)$$

As for COP, a solution of a MOCO problem is a complete assignment of its decision variables such that its set of constraints is satisfied. The decision space of a MOCO problem is formally defined as follows:

Definition 2.1 (Decision Space). Let $\mathcal{P}$ be a multi-objective combinatorial optimization problem with a set of n decision variables $vars(\mathcal{P}) = \{x_1, \dots, x_n\}$. Its decision space $\mathcal{S}$ is defined as the cartesian product of the domain of each decision variable $\mathcal{S} = D(x_1) \times \dots \times D(x_n)$.

For convenience, solutions of the decision space can be projected into the objective space. The objective space is defined as follows:

Definition 2.2 (Objective Space). Let $\mathcal{P}$ be a multi-objective combinatorial optimization problem with a set of m objectives $objs(\mathcal{P}) = \{obj_1, \dots, obj_m\}$. Its objective space $\mathcal{S}^{obj}$ is defined as the cartesian product of the domain of each objective $\mathcal{S}^{obj} = D(obj_1) \times \dots \times D(obj_m)$.

In general, the decision space and the objective space are represented as hyper-spaces in which solutions are vectors. This representation is particularly convenient for bi-objective problems.¹

Definition 2.3 (Solution vector). Let x be a variable of a MOCO problem $\mathcal{P}$ with a solution sol . The value $sol(x)$ denotes the value assigned to x in the solution sol . The projection of sol in the decision space corresponds to the vector $(sol(x_1), \dots, sol(x_n)) \in \mathcal{S}$. The projection of sol in the objective space corresponds to the vector $(sol(obj_1), \dots, sol(obj_m)) \in \mathcal{S}^{obj}$.

2.1.1 Pareto-dominance

One difficulty of multi-objective problems is that the conflicting nature of the objectives usually prevents the existence of a solution sol^* that is optimal for all the objectives [54]:

$$\forall sol \in sols(\mathcal{P}), \forall i \in \{1, \dots, m\} : sol^*(obj_i) \leq sol(obj_i)$$

To address this problem, a new partial ordering relation known as *Pareto-dominance* (or *dominance*) has been established [40]. It is defined as follows:

Definition 2.4 (Pareto-dominance). A solution sol_1 *dominates* a solution sol_2 , denoted by $sol_1 \prec sol_2$, if and only if:

$$\begin{aligned} & \forall j \in \{1, \dots, m\} : sol_1(obj_j) \leq sol_2(obj_j) \\ & \wedge \exists j \in \{1, \dots, m\} : sol_1(obj_j) < sol_2(obj_j) \end{aligned}$$

The notion of Pareto-optimality directly follows from the definition of this relation:

Definition 2.5 (Pareto-optimality). Consider a MOCO problem $\mathcal{P}$. A solution sol^* is *Pareto-optimal* if and only if there exists no other solution $sol' \in sols(\mathcal{P})$ such that $sol' \prec sol^*$.

In other terms, a solution is said to be Pareto-optimal if it is impossible to improve the value of one objective without degrading the value of at least one other objective.

Definition 2.6 (Pareto-equivalence). Two solutions sol_1 and sol_2 are *Pareto-equivalent* if they do not dominate each other: $sol_1 \not\prec sol_2 \wedge sol_2 \not\prec sol_1$.

Observe that Pareto-optimal solutions are always Pareto-equivalent to themselves. These notions are illustrated by Figure 2.1 where:

- Solutions a , c , and e are Pareto-optimal;
- The solution c dominates the solutions b and d ;
- The solution e dominates the solution d ;
- Solutions a , b and d are Pareto-equivalent.

¹We choose to illustrate the definitions of this section on bi-objective problems.

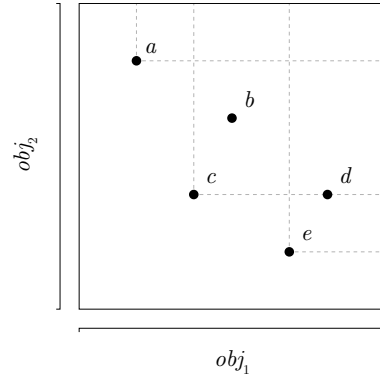


Figure 2.1: An arbitrary set of solutions in a bi-objective space. Solutions a , c , and e are Pareto optimal.

2.1.2 Pareto front

In Multi-objective optimization, one is generally interested to discover the set of all Pareto-optimal solutions known as the *Pareto set*.

Definition 2.7 (Pareto set). Consider a MOCO problem $\mathcal{P}$. Its *Pareto set* is the set off all the Pareto-optimal solutions of the problem:

$$\{sol \mid sol \in sols(\mathcal{P}), \nexists sol' \in sols(\mathcal{P}) : sol' \prec sol\}$$

The Pareto set can be projected in the objective space to form the *Pareto front*. Figure 2.2 illustrates the Pareto set and the Pareto front of an arbitrary problem with two decisions variables. Observe that solutions that are close in the objective space are not guaranteed to be close in the decision space.

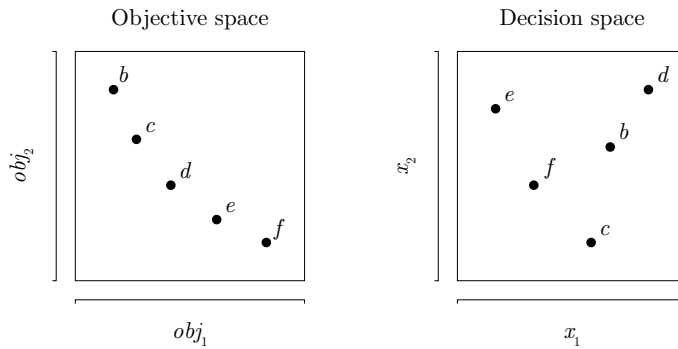


Figure 2.2: The Pareto front (left) and the Pareto set (right) of an arbitrary problem with two decision variables. Solutions that are close in the objective space are not guaranteed to be close in the decision space.

2.1.3 Weak and Strong dominance

One particularity of the Pareto-dominance is that a solution does not dominate itself. Hence, a Pareto-front based on the Pareto-dominance relation may

contain different solutions in the decision space that have a same projection in the objective space. Definitions 2.8 and 2.9 introduce weaker and stronger relations to order multi-objective solutions. The difference between these relations is illustrated in Figure 2.3.

Definition 2.8 (Weak Pareto-dominance). A solution sol_1 *weakly dominates* a solution sol_2 , denoted by $sol_1 \preceq sol_2$, if and only if:

$$\forall j \in \{1, \dots, j\} : sol_1(obj_j) \leq sol_2(obj_j)$$

Definition 2.9 (Strong Pareto-dominance). A solution sol_1 *strongly dominates* a solution sol_2 , denoted by $sol_1 \prec sol_2$, if and only if:

$$\forall j \in \{1, \dots, j\} : sol_1(obj_j) < sol_2(obj_j)$$

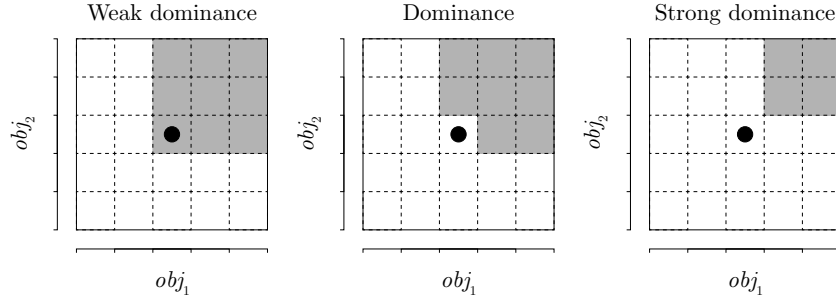


Figure 2.3: Differences between weak Pareto-dominance, Pareto-dominance, and strong Pareto-dominance in a bi-objective space. Hatched lines represent the discretization of the bi-objective space. Grey areas corresponds to dominated solutions.

Given a multi-objective problem $\mathcal{P}$, its Pareto front based on weak Pareto-dominance ($\preceq$) is equivalent to its Pareto front based on Pareto-dominance ($\prec$). Still, weak Pareto-dominance only keeps one solution for a given projection in the objective space. Therefore, the Pareto set based on weak Pareto-dominance may contain less solutions than the one based on standard Pareto-dominance. Pareto sets based on weak Pareto-dominance are called *minimal Pareto sets*.

2.2 Archive and quality

Discovering the exact Pareto set implies finding all the Pareto-optimal solutions and may be impracticable on large MOCO problems. Hence, a large class of MOCO algorithms focus on finding the best *archive* that is an approximation of the Pareto set.

Definition 2.10 (Archive). An archive $\mathcal{A} = \{sol_1, \dots, sol_n\}$ is a set of Pareto-equivalent solutions:

$$\forall sol \in \mathcal{A}, \nexists sol' \in \mathcal{A} : sol' \prec sol$$

As illustrated on Figure 2.4, an archive partitions the objective space into three subspaces:

Definition 2.11 (Dominated subspace). The *dominated subspace* consists of all the solutions that are dominated by at least one solution in the archive.

Definition 2.12 (Diversification subspace). The *diversification subspace* consists of all the solutions that neither dominate nor are dominated by any solution in the archive.

Definition 2.13 (Intensification subspace). The *intensification subspace* consists of all the solutions that dominate at least one solution in the archive.

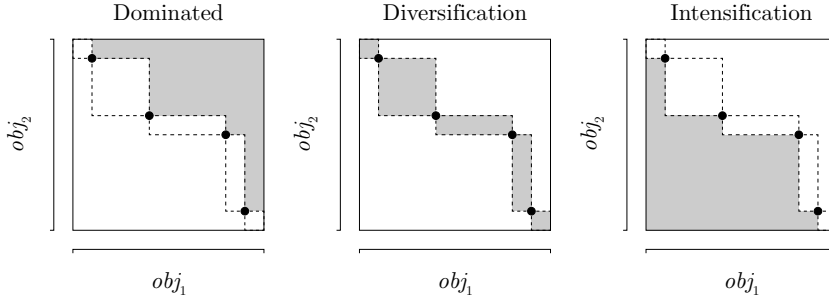


Figure 2.4: An archive partitions the objective space into three subspaces: (a) the dominated subspace, (b) the diversification subspace, and (c) the intensification subspace.

Clearly the archive quality can only be improved by adding new solutions from the intensification subspace where a new solution replaces at least one solution in the archive (Figure 2.5 left); or from the diversification subspace where a new solution is added into the archive without replacing any other solution (Figure 2.5 right).

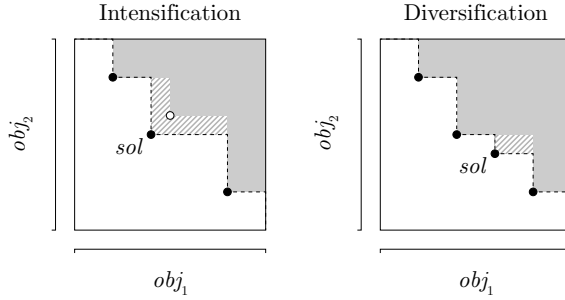


Figure 2.5: Improvement of an archive by adding a new solution from the intensification subspace (left) and the diversifications subspace (right). Hatched areas correspond to the increase of the dominated subspace. The white solution is removed from the archive.

Difficulties arise when two archives have to be compared. Indeed, it is not likely that an archive $\mathcal{A}_1$ completely dominates another archive $\mathcal{A}_2$ (see Figure 2.6). Particularly, some solutions of $\mathcal{A}_1$ may dominate solutions of $\mathcal{A}_2$ and vice-versa. As well, some solutions of $\mathcal{A}_1$ and $\mathcal{A}_2$ may be Pareto-equivalent.

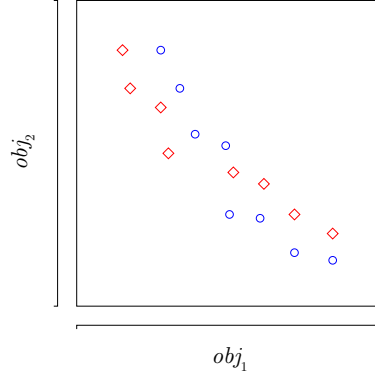


Figure 2.6: Two different arbitrary archives. Determining which archive is better than the other is not a trivial task.

Nowadays, many quality indicators, with different properties, have been proposed to measure the quality of an archive [25, 36, 62]. However, the choice of a global quality indicator is still in open discussions. A list of the most famous indicators is proposed in [54]. Among them, the hypervolume indicator $\mathcal{H}$ is an unary quality indicator (to be maximized) which measures the volume of the objective subspace dominated by a given archive [64] *i.e.* the dominated subspace (Definition 2.11). The hypervolume indicator has the interesting property of being strictly monotonic according to the Pareto dominance relation. That is, given Φ the set of all the possible archives:

$$\forall \mathcal{A}, \mathcal{A}' \in \Phi, \mathcal{A} \prec \mathcal{A}' : \mathcal{H}(\mathcal{A}) > \mathcal{H}(\mathcal{A}')$$

This indicator is mostly used for bi-objective problems since its computational complexity increases exponentially with the number of objectives [60]. Figure 2.7 shows the hypervolume of both archives from Figure 2.6.

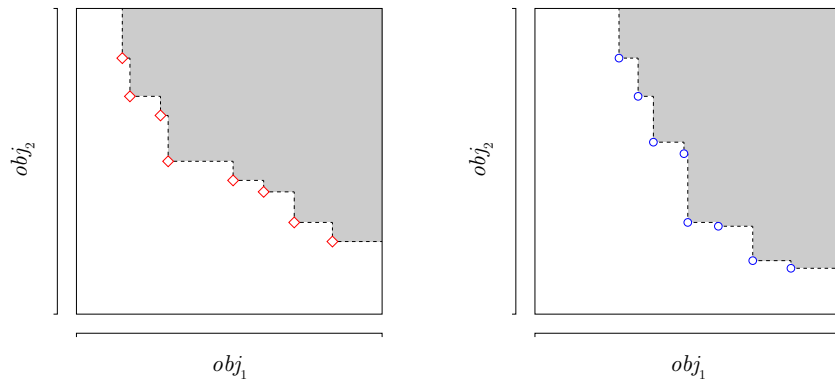


Figure 2.7: Hypervolume $\mathcal{H}$ of two archives. The archive on the left has the largest hypervolume. It is thus better, according to the hypervolume indicator, than the archive on the right.

2.3 Non-supported solutions

All the solutions of the Pareto front are not equally difficult to discover. For instance, a *supported solution* [14] is a Pareto optimal solution that can be discovered by minimizing a linear aggregation of the objectives:

$$\min_{sol \in \text{sols}(\mathcal{P})} \left\{ \sum_{i=1}^m \lambda_i \cdot \text{sol}(\text{obj}_i) \right\}, \quad \sum_{i=1}^m \lambda_i = 1$$

In other words, a supported Pareto optimal solution is an extreme point on the convex hull of the Pareto front. Supported solutions can be easily discovered using single-objective optimization approach. Nevertheless, there exists other Pareto optimal solutions, known as *non-supported solutions*, that cannot be discovered using a linear aggregation of objectives (see Figure 2.8). Hence, finding the non-supported solutions justifies the needs of more advanced techniques to tackle multi-objective combinatorial optimization problems.

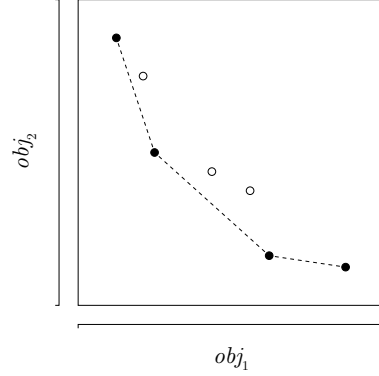


Figure 2.8: Difference between supported solution (in black) and non-supported solutions (in white). Supported solutions correspond to extreme points on the convex hull of the Pareto front.

2.4 Multi-Objective Problems

To conclude this introduction to multi-objective combinatorial optimization, we present three MOCO problems that are studied in this master's thesis. First both problems are classical academic problems well studied in the MOCO research community. The last one is a real-world complex bi-objective optimization problem known as the bi-objective tank allocation problem.

2.4.1 Multi-Objective Knapsack Problem

The Multi-Objective 0/1 Knapsack Problem (MOKP) consists of a set of n items, a set of m profits, and a maximal capacity C . Each item i is associated with a positive weight w_i and a non negative vector $p_i = (p_{i1}, \dots, p_{im})$ where p_{ij} is the contribution of item i to profit j . A solution to this problem is represented by the vector $x = (x_1, \dots, x_n)$ where x_i is a binary decision variable equal to 1

if the item i is assigned to the knapsack and 0 otherwise. The problem to solve is defined as follows:

$$\begin{aligned} \text{Maximize} \quad & obj = (obj_1, \dots, obj_m) \\ \text{Subject to} \quad & obj_j = \sum_{i=1}^n x_i \cdot p_{ij} \\ & \sum_{i=1}^n x_i \cdot w_i \leq C \end{aligned}$$

2.4.2 Multi-Objective Quadratic Assignment Problem

The single-objective quadratic assignment problem consists in assigning n facilities to n different locations while minimizing the costs of the flows (*e.g.* amount of supplies) between these facilities.

More formally, let $x(i)$ be the position of facility i , $d_{x(i)x(j)}$ be the distance between the location of facilities i and j , and w_{ij} be the flow between these facilities. The objective to minimize is the sum of the product of the flows and the distance between the facilities:

$$\sum_{i=1}^n \sum_{j=1}^n w_{ij} \cdot d_{x(i)x(j)}$$

The Multi-Objective Quadratic Assignment Problem (MOQAP) with m weight matrices naturally models any facility layout problem where we are concerned with the flow of more than one type of item or agent [29]:

$$\begin{aligned} \text{Minimize} \quad & obj = (obj_1, \dots, obj_m) \\ \text{Subject to} \quad & obj_l = \sum_{i=1}^n \sum_{j=1}^n x_{ij}^l \cdot d_{x(i)x(j)} \end{aligned}$$

2.4.3 Bi-Objective Tank Allocation Problem

The bi-Objective Tank Allocation Problem (bTAP) [48] involves the assignment of different cargoes (volumes of chemical products to be shipped by the vessel) to the available tanks of the vessel while satisfying hard segregation constraints *e.g.* to avoid placing dangerous cargoes in adjacent tanks.

An ideal loading plan should maximize the total volume of unused tanks (*i.e.* free space) to minimize cleaning costs. Minimizing the number of used tanks is also desirable in order to maximize the chances of accommodating other cargoes in next visited ports. Therefore, the objectives of the problem are as follows:

- Maximize the volume of unused tanks;
- Maximize the number of unused tanks.

A complete description of the bi-objective tank allocation problem is presented in [48].

Chapter 3

Related Work

This chapter describes the related work in applying constraint programming to the field of multi-objective combinatorial optimization. The first approach was initially proposed to solve bi-objective combinatorial optimization problems with complete methods as mixed integer programming or, of course, constraint programming (Section 3.1). The second approach, originally developed for interval constraint satisfaction problems, aims at providing good approximation of the lower and upper bound of the exact Pareto front (Section 3.2). Third approach adapts the branch-and-bound search of constraint programming to solve multi-objective combinatorial problems in one search by removing dominated branches of the search tree (Section 3.3).

3.1 A complete bi-objective algorithm

A particularity of bi-objective optimization is that improving the value of the first objective of a Pareto optimal solution cannot be done without degrading the value of its second objective. This property is directly derived from the definition of Pareto optimality (Definition 2.5). The approach proposed by van Wassenhove and Gelders [59] exploits this property to find the minimal Pareto fronts of bi-objective optimization problems. The algorithm is as follows:

1. Find the optimal solution sol according to obj_1 ;
2. Generate a new CSP with the additional constraint $obj_2 < sol(obj_2)$;
3. Search for the optimal solution sol according to obj_1 ;
4. If this solution exists, go back to step 2; else stop.

This iterative process terminates when the problem becomes over-constrained *i.e.* it is unfeasible. Therefore, the approach of van Wassenhove decomposes the problem in a sequence of more and more constrained subproblems.¹ A sequence of the generated subproblems is depicted in Figure 3.1.

¹The approach of van Wassenhove and Gelders may be seen as a particular instance of the ϵ -constraint method introduced in [24].

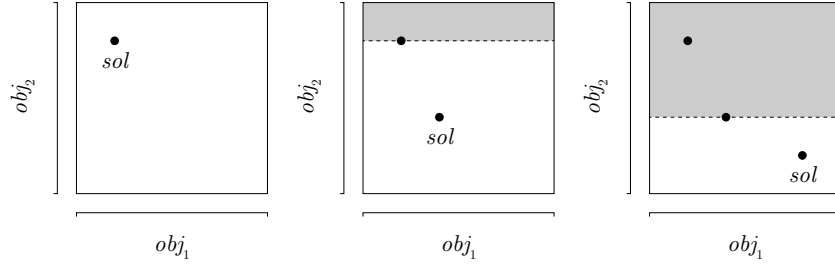


Figure 3.1: The first three iterations of the van Wassenhove's algorithm. The hatched lines correspond to the constraints $obj_2 < obj_2(sol)$.

3.2 PICPA

The Population and Interval Constraint Propagation Algorithm (PICPA) [3] aims at tackling constrained continuous multi-objective optimization problems by combining constraint programming with evolutionary concepts *e.g.* population. As well, PICPA can be adapted to tackle multi-objective combinatorial optimization problems. The particularity of PICPA is to provide users with bounds on the exact Pareto front as well as an approximation of the Pareto front.

3.2.1 Boxes and Box spaces

PICPA maintains a population of non-overlapping boxes. Each box represents a part of the objective space which could contain Pareto optimal solutions. The union of all the boxes in the population provides an approximation of the Pareto front (proofs are provided in [3]). A *box* and its corresponding *box space* are defined as follows:

Definition 3.1 (Box). A *box* is a vector of intervals $B = ([b_1, b'_1], \dots, [b_n, b'_n])$.

Definition 3.2 (Box space). Given a box $B = ([b_1, b'_1], \dots, [b_n, b'_n])$, its box space B_S is the set of the vectors $b \in \mathbb{R}^n$ contained in the cartesian product of the intervals $[b_i, b'_i]$: $B_S = \{b \in \mathbb{R}^n \mid b \in [b_1, b'_1] \times \dots \times [b_n, b'_n]\}$.

Hence, a box defined as the vector of the interval domains of the objective variables (see Definition 1.9) provides a box space that contains all the solution of the objective space:

$$\forall sol \in \mathcal{S}^{obj} : sol \in B_S, B = ([D(obj_1)], \dots, [D(obj_m)])$$

Particularly, the intervals of a box $B = ([b_i, b'_i], \dots, [b_m, b'_m])$ can be used to constrain the objective variables of the problem to solve:

$$\forall i \in \{1, \dots, m\} : b_i \leq obj_i \leq b'_i$$

Such a box thus corresponds to a subproblem of the problem to solve.

The following definition introduces the *PS-dominance* relation to compare solution with boxes. Figure 3.2 illustrates this new relation.

Definition 3.3 (PS-dominance). Given a MOCO problem $\mathcal{P}$ with m objectives, a solution sol of $\mathcal{P}$, and a box $B = ([b_1, b'_1], \dots, [b_m, b'_m])$, we said that sol PS-dominates B (denoted $sol \prec_{PS} B$) if and only if $sol \prec ([b_1], \dots, [b_m])$. Conversely, B PS-dominates sol (denoted $B \prec_{PS} sol$) if and only if $([b'_1], \dots, [b'_m]) \prec sol$.

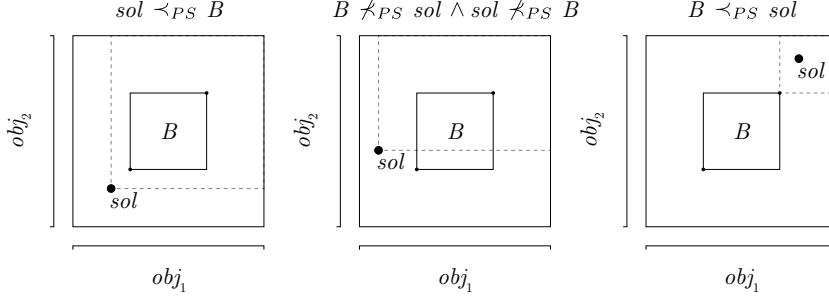


Figure 3.2: Illustration of the PS-dominance relation ($\prec_{PS}$) to compare solutions and boxes.

3.2.2 Bisection-Contraction

PICPA starts by initializing its population with one individual box B defined as the vector of the interval domains of the objective variables. Clearly, this first box represents the objective space of the problem to solve.

Then, PICPA selects an objective obj_i and bisects its interval $[b_i, b'_i]$ in B leading to two new individuals B' and B'' :

$$B' = ([b_1, b'_1], \dots, [b_{i-1}, b'_{i-1}], [b_i, v - 1], [b_{i+1}, b'_{i+1}], \dots, [b_m, b'_m])$$

$$B'' = ([b_1, b'_1], \dots, [b_{i-1}, b'_{i-1}], [v, b'_i], [b_{i+1}, b'_{i+1}], \dots, [b_m, b'_m])$$

Such a bisection triggers two propagation processes to contract the boxes by strengthening the domain of each objective. Both new contracted individuals B' and B'' replace their parent B in the population. Notice that contracted boxes with an empty interval (*i.e.* objective with an empty domain) are not inserted in the population. This iterative bisection-contraction process continues until the population reaches its fixed maximum size. Figure 3.3 illustrates one iteration of the bisection-contraction process.

3.2.3 Instantiation and box elimination

The instantiation process starts once the population has reached its maximal size. It aims at instantiating each box B of the population into a complete satisfiable solution sol such that $sol \in B_S$. Each time a solution is successfully instantiated, it is added into a second population of Pareto equivalent solutions *i.e.* an archive. Notice that instantiating a box may be computationally intensive. Therefore, PICPA limits the search effort to instantiate each box.

The next step is to use the instantiated solutions to eliminate PS-dominated boxes from the box population. As the population is reduced by this elimination

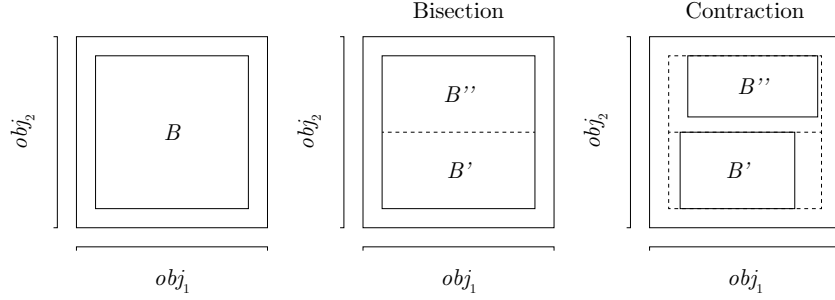


Figure 3.3: Illustration of the bisection-contraction process on an arbitrary box. The box B is bisected on objective obj_2 and lead to two boxes B' and B'' that are contracted using propagation.

process, the algorithm restarts the above bisection-contraction process to extend the population to its maximal size. Figure 3.4 illustrates the elimination process. PICPA terminates if one of the following two cases is met:

1. The population becomes empty meaning that the problem is unsatisfiable;
2. The elimination process cannot remove any individual in the box population.

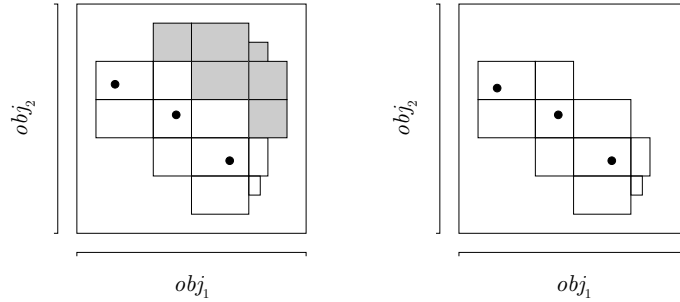


Figure 3.4: Illustration of the elimination process. Grey boxes are PS-dominated by the instantiated solutions and have to be eliminated.

3.2.4 Discussion

PICPA was originally designed to solve continuous multi-objective optimization problems. Results on simple academic continuous problems are presented in [3]. However, our preliminary experiences showed that the adaptation of PICPA to multi-objective combinatorial optimization problems is clearly outperformed by the algorithm of van Wassenhove presented above (Section 3.1).

3.3 Multi-objective branch and bound

Gavanelli [19] suggested a framework to find the exact Pareto front of multi-objective combinatorial optimization problems within a single constraint pro-

gramming search. This framework is presented as a specialized branch-and-bound search making use of previously discovered solutions to prune weakly-dominated branches of the search tree *i.e.* branches that cannot lead to a Pareto optimal solution. Each time a new solution is encountered, a new “nogood” constraint is added to enforce the upcoming solutions to not be weakly-dominated by this new solution. Therefore, the search is guaranteed to discover only non-dominated solution according to the maintained archive.

Apart from being more general, this framework has been demonstrated to be more efficient than the van Wassenhove and Gelders algorithm (Section 3.1) to solve bi-objectives knapsack problems [19].²

3.3.1 Filtering rule

The filtering rule presented by Gavanelli considers the extremities of the objective space (represented as a box) to determine if an objective variable has to be reduced. Let obj_i^{min} and obj_i^{max} denote the minimal and maximal value of obj_i , the *dominated point* DP_i is defined as follows:

$$DP_i = (obj_1^{min}, \dots, obj_{i-1}^{min}, obj_i^{max}, obj_{i+1}^{min}, \dots, obj_m^{min}).$$

The domain of the objective variable obj_i can only be reduced if the dominated point DP_i is weakly-dominated by an already discovered solution *i.e.* a solution in the archive $\mathcal{A}$:³

$$\exists sol \in \mathcal{A}, sol \preceq DP_i : obj_i^{max} \leftarrow sol(obj_i) - 1.$$

Therefore, a reduction of the objective space only occurs if one of the points $DP_1, \dots, DP_m$ lies in the dominated subspace. This proposition is illustrated in Figure 3.5.

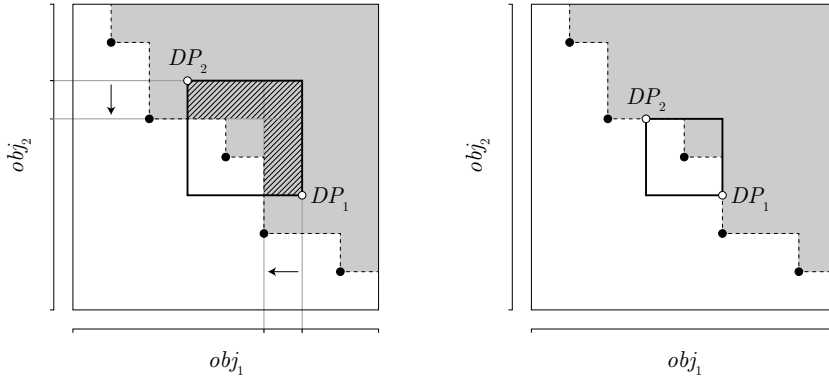


Figure 3.5: The objective space before and after applying Gavanelli’s proposition. Hatched areas correspond to parts of the objective space that only contain dominated solutions w.r.t. the archive.

²This is probably due to the fact that the approach of Gavanelli does not need to restart the search at each discovered solution. Besides, non-optimal solutions already provide some pruning power that can be used to remove dominated branches of the search tree.

³Proofs of this proposition are presented in [19].

3.3.2 Complexity

At each iteration, the filtering procedure proposed by Gavanelli has to search for solutions in the archive that may dominate the dominated points $DP_1, \dots, DP_m$. Hence, the complexity of the filtering procedure is strongly related to the time needed to find such solutions.

In his work, Gavanelli proposed to use *quad-trees*⁴ to implement the archive (quad-trees are described in depth in Section 7.3). Following Gavanelli, quad-trees allow to determine if a solution is dominated by a solution from the archive with a time complexity of $\mathcal{O}(\log n)$ where n is the number of solutions in the archive. Unfortunately, this statement only holds for the specific case where the quad-tree is bi-dimensional and balanced. This is a strong assumption since standard quad-trees are unbalanced data structures. Therefore, the time complexity of $\mathcal{O}(n)$ seems more accurate. Besides, supposing that the quad-tree is balanced, the actual worst case complexity of the access operation may be higher than $\mathcal{O}(\log n)$ when considering more than two objectives. The actual worst case complexity is defined by the following formula:

$$\mathcal{O}\left(\sum_{i=0}^{\log_a n} \left(\frac{a}{2}\right)^i\right) \quad \text{with} \quad a = 2^m - 2$$

which is strictly higher than $\mathcal{O}(\log n)$ when $m > 2$.⁵

⁴Quad-trees can be seen as the extension of binary trees in multi-dimensional spaces.

⁵See Section 7.3.1 for details on this formula.

Chapter 4

Multi-Objective Large Neighborhood Search

Multi-Objective Large Neighborhood Search (MO-LNS) is a population based framework which hybridizes constraint programming and local search to find good approximations of the Pareto front while keeping a declarative constraint programming model. MO-LNS extends the well known LNS framework introduced by Shaw [50] (Section 1.7) to the field of multi-objective combinatorial optimization problems. Instead of maintaining a best-so-far solution, MO-LNS maintains a best-so-far approximation of the Pareto front *i.e.* an archive. At each iteration, one solution of the archive is selected, relaxed, and optimized to strictly improve the hypervolume of the archive. The MO-LNS framework is summarized as follows:

1. Find an initial set of solutions and insert it in the archive.
2. Repeat until a stop criterion is met:
 - 2.1. Randomly select an objective to drive the iteration.
 - 2.2. Choose to diversify or to intensify the archive.
 - 2.3. Select a solution in the archive.
 - 2.4. Generate a new neighborhood by relaxing the selected solution.
 - 2.5. Explore the generated neighborhood with constraint programming.
 - 2.6. Add all the discovered solutions into the archive.

One particularity of MO-LNS is that it allows users to reuse efficient single-objective relaxation procedures and search heuristics to solve multi-objective problems. Therefore, only two decisions have to be taken to implement the MO-LNS framework (steps 2.2 and 2.3).

This chapter describes and details the main components of MO-LNS. The first section is dedicated to the **Pareto** constraint, that is a new global constraint enforcing the discovery of nondominated solutions w.r.t. to the maintained archive. Section 4.2 presents the different optimization modes that can be used to improve the archive (step 2.2). Finally, Section 4.3 addresses the problem of selecting the solution that will be used to generate the neighborhood explored using constraint programming (step 2.3).

4.1 The Pareto constraint

The idea of the **Pareto** constraint is to use the filtering rule introduced by Gavanelli [19] (see Section 3.3) as the filtering rule of a new global constraint defined over the objective variables $obj_i, \dots, obj_m$ and an archive $\mathcal{A}$:

$$\text{Pareto}(obj_1, \dots, obj_m, \mathcal{A} = \{sol_1, \dots, sol_n\})$$

The **Pareto** constraint aims at pruning branches that are weakly-dominated ($\preceq$) by the previously discovered solutions *i.e.* the solutions contained in the archive. Therefore, the constraint ensures that each new discovered solution is nondominated w.r.t. the archive:

$$\nexists sol \in \mathcal{A} : sol \preceq (obj_1, \dots, obj_m)$$

New solutions can thus be inserted into the archive, strengthening the filtering of the **Pareto** constraint.

4.1.1 Filtering

The filtering process of the **Pareto** constraint aims at reducing the upper bound of the objective variables. To know if the domain of an objective variable obj_i has to be reduced, the filtering process considers its corresponding dominated point DP_i that is defined as follows:

$$DP_i = (obj_1^{\min}, \dots, obj_{i-1}^{\min}, obj_i^{\max}, obj_{i+1}^{\min}, \dots, obj_m^{\min})$$

If the dominated point DP_i is weakly-dominated by a solution in the archive, we can use this solution to adjust the upper bound of the objective:

$$\exists sol \in \mathcal{A}, sol \preceq DP_i : obj_i^{\max} \leftarrow sol(obj_i) - 1 \quad (4.1)$$

This filtering rule has to be executed until there exists no solution in the archive that dominates the dominated point. Therefore, the filtering algorithm of the **Pareto** constraint has to be called several times to reach its fix point. This situation is illustrated in Figure 4.1 where the solutions of the archive are selected in the worst possible order.

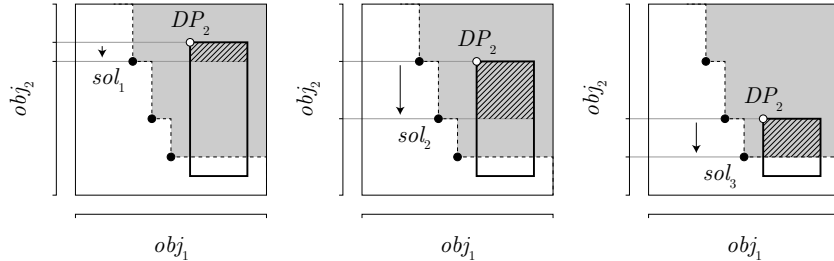


Figure 4.1: A dominated point DP_i may be dominated by several solutions. The order in which solutions are selected influence the time spent to reach the fix point of the **Pareto** constraint.

From the filtering rule defined in Equation 4.1, it appears that the constraint can reach its fix point in one step (*i.e.* the constraint becomes idempotent) if it is able to access directly the solution that dominates DP_i with the tightest value for the objective variable obj_i . Hence, the filtering rule becomes:

$$obj_i^{\max} \leftarrow \min(\{obj_i^{\max}\} \cup \{sol(obj_i) - 1 \mid sol \in \mathcal{A} \wedge sol \preceq DP_i\}) \quad (4.2)$$

Example 4.1. Consider the constraint $\text{Pareto}(obj_1, obj_2, obj_3, \mathcal{A})$ with:

$$\begin{aligned} D(obj_1) &= \{3, \dots, 5\}, \\ D(obj_2) &= \{2, \dots, 5\}, \\ D(obj_3) &= \{2, \dots, 5\}, \\ \mathcal{A} &= \{(1, 4, 2), (4, 2, 3), (2, 3, 1), (2, 1, 4)\}. \end{aligned}$$

No filtering for $obj_1^{\max}$ is possible because the dominated point $DP_1 = (5, 2, 2)$ is not dominated by any solution in $\mathcal{A}$. For obj_2 some filtering is possible since $DP_2 = (3, 5, 2)$ is dominated by $(1, 4, 2)$ and $(2, 3, 1)$. We can set $obj_2^{\max} \leftarrow \min(4-1, 3-1) = 2$. The domain of obj_3 can also be filtered since $DP_3 = (2, 2, 5)$ is dominated by $(2, 1, 4)$. We can thus set $obj_3^{\max} \leftarrow 4 - 1 = 3$.

4.1.2 Complexity

At each execution, the Pareto constraint has to check the dominated points $DP_1, \dots, DP_m$ to reduce the domain of each objective variable. Therefore, the filtering algorithm of the idempotent Pareto constraint has a time complexity of $m\mathcal{O}(\text{access time})$ where “access time” is the time complexity of accessing the *tightest solution* (Equation 4.2).

Using a well defined data structure, it is possible to access tightest solutions of bi-objective problems with a worst case time complexity of $\mathcal{O}(\log n)$ (see Section 7.2). Hence, the time complexity of the filtering algorithm is $\mathcal{O}(2 \log n)$.

Difficulties arise when the number of objectives is higher than two (see Section 7.3). By making the assumption that quad-trees are balanced, it is possible to access all the tightest solutions with a time complexity of:

$$m\mathcal{O}\left(\sum_{i=0}^{\log_a n} \left(\frac{a}{2}\right)^i\right) \quad \text{with} \quad a = 2^m - 2$$

Unfortunately, quad-trees are not auto-balanced and the actual worst case time complexity of accessing all the tightest solutions is $\mathcal{O}(mn)$. As a consequence, we suggest to access the m tightest solutions in a single sweep over all the solutions contained in the archive. Despite a guaranteed time complexity of $\Theta(mn)$, this approach is far much easier to implement (see Section 7.1).

4.1.3 MO-CP versus MO-LNS

Constraint programming alone could be used to solve multi-objective combinatorial optimization problems by enumerating all the solutions and keeping the Pareto optimal ones. Clearly, this approach presents some practical issues due to the exponential number of solutions to consider (see Section 1.2.1). The Pareto constraint (introduced in Section 4.1) offers a considerable speed-up to

constraint programming by pruning branches that are dominated by the already discovered solutions. This approach mixing constraint programming with the **Pareto** constraint is referenced as Multi-Objective Constraint Programming (MO-CP) in the remainder of this master’s thesis.

Despite the speed-up offered by the **Pareto** constraint, MO-CP still suffers from significant scalability issues. Indeed, it is not likely that a partial assignment – even of a single variable – appears in all the solutions of the Pareto front. Therefore, the choices made in the first levels of the search tree will always have to be reconsidered. This leads to some important diversification issues as well as a weaker filtering of the **Pareto** constraint. Alternative search algorithms as *discrepancy search* (Section 1.4.3) may be used to address this problem. Unfortunately, *discrepancy search* suffers from its exponential space complexity that often prevents the exploration of the whole search tree.

MO-LNS addresses the scalability issues of MO-CP by exploring various neighborhood generated from different solutions.¹ This allows MO-LNS to explore very diversified regions of the problem’s decision space. As well, MO-LNS uses its optimization modes to provide the Pareto constraint with a well diversified and intensified archive in a few iterations.

4.2 Optimization modes

The goal of MO-LNS is to provide users with good approximations of the Pareto front. These approximations are represented as an archive that is iteratively improved by MO-LNS. We distinguish two ways of improving an archive:

1. finding a new solution in the *intensification subspace* (Definition 2.13). We call this an *intensification* of the archive. The resulting archive is not larger after this insertion since some solutions may disappear from the archive.
2. finding a new solution in the *diversification subspace* (Definition 2.12). We call this a *diversification* of the archive. The resulting archive has one more element;

Observe that both improvements strictly increase the hypervolume. As well, both improvements are allowed by the **Pareto** constraint which guarantees that only new nondominated solutions can be discovered.

The discovery of a new solution may contribute to diversify the archive, or it may intensify it. Each of these new solutions is used by the Pareto constraint to strengthen its pruning power. Hence, a good strategy in term of filtering consists in finding quickly a limited number of solutions that cover the largest part of the dominated subspace *i.e.* a well diversified set of solutions close to the Pareto front. Conversely, it would be less efficient to quickly discover a large number of nondominated solutions while being far from the Pareto front. Those two situations are illustrated in Figure 4.2.

An archive with many solutions quite far from the exact Pareto front is often the result of too many diversification at the early iterations of MO-LNS. It is thus important to have a good trade-off between diversification and intensification. A first idea to control the ratio of diversification/intensification

¹Chapter 6 is dedicated to the difference of performance between MO-LNS and MO-CP.

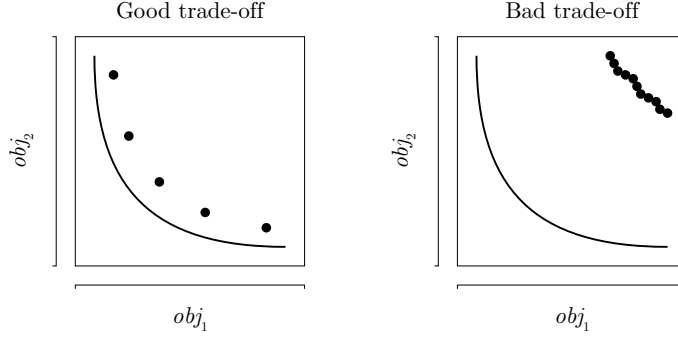


Figure 4.2: Good and bad diversification/intensification trade-off in the early iterations of MO-LNS. The Pareto front is represented by the plain curve.

is to adapt the search heuristic dynamically. One could for instance have two different search heuristics, one that favors intensification and the other one favoring diversification. This approach has the main disadvantage of requiring a good knowledge of the problem and an additional implementation work by the modeler. An alternative and more general approach forces diversification or intensification at each restart, based on a change of the filtering behavior of the different objectives.

4.2.1 Variable-Objective

The idea of dynamically changing the filtering of the objectives has been proposed in [47]. Each objective can be set independently into three different filtering modes during the branch and bound search:

1. *No-Filtering*: the filtering of the objective is disabled, having no impact at all.
2. *Weak-Filtering*: each time a new solution is discovered during the search, the bound of the objective is updated such that the next discovered solution has a better or equal bound for this objective.
3. *Strong-Filtering*: each time a new solution is discovered during the search, the bound of the objective is updated such that the next discovered solution strictly improves the bound of this objective.

We introduce a fourth filtering mode to help us controlling the trade-off between diversification and intensification:

4. *Maintain-Filtering*: the bound of the objective is not updated during the search, but it has to be strictly lower than a given value.

4.2.2 Intensification

The goal of intensification is to discover new solutions that dominate the selected one. Hence, the intensification process focuses on the intensification subspace. A simple way to guarantee the discovery of such solutions is to constrain the search on the part of the intensification space that contains this kind of solutions. We propose both following intensification processes:

Driven intensification. In *driven intensification*, the selected objective variable obj_i is configured in *Strong-Filtering* mode. The other objective variable obj_j ($j \neq i$) are configured in *Weak-Filtering* mode. Hence, driven intensification enforces the discovery of solution that dominate the selected one.

Strong intensification. In *strong intensification*, all the objective variable are set in *Strong-filtering* mode. Hence, each objective is constrained to be strictly lower than its value in the selected solution. As a consequence, strong intensification only discover solutions that strongly-dominates the selected one. Notice that *strong intensification* does not rely on a selected objective.

Both intensification process are illustrated in Figure 4.3.

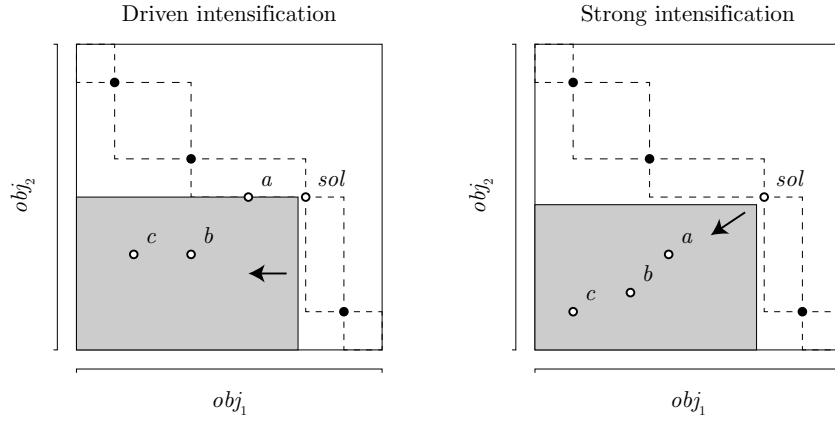


Figure 4.3: Intensification. (left) obj_1 is set into the *Strong-Filtering* mode and obj_2 is set into the *Weak-Filtering* mode. (right) obj_1 and obj_2 are both set into *Strong-Filtering* mode. For both configurations, a possible sequence of successive discovered solutions is given.

4.2.3 Diversification

The diversification process focuses on the diversification subspace. It aims at increasing the cardinality of the archive by finding new solutions that are neither dominating nor dominated by any solution already in the archive. Nonetheless, the diversification process is also able to discover new solutions that dominate some solutions in the archive.

Naive approach

The diversification process was initially designed to ensure the discovery of nondominated solutions without using the **Pareto** constraint. The idea is to use the selected objective to consider a part of the diversification subspace that is locally adjacent to the selected solution.

More formally, let sol be the selected solution and $i \in \{1, \dots, m\}$ be the selected objective. The search space to explore is defined by setting the objective variable obj_i in *Strong-Filtering* mode, and by forcing each other objective

variable obj_j ($j \neq i$) to be strictly lower than its closest upper bound u_j :

$$u_j = \min(\{sol'(obj_j) \mid sol' \in \mathcal{A} \wedge sol'(obj_j) > sol(obj_j)\} \cup \{obj_j^{max}\}) \quad (4.3)$$

The objective variables obj_j are configured in *Maintain-Filtering* mode and ensure that the value of the objective j is strictly lower than u_j . Figure 4.4 illustrates the naive diversification process on an arbitrary bi-objective archive. As we can see, this first diversification process makes the naive assumption that solutions that are close in the objective space are also close in the decision space.

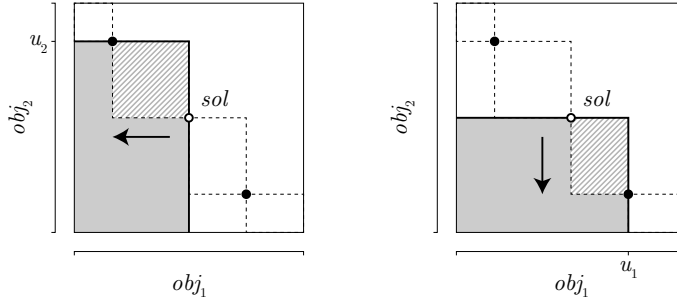


Figure 4.4: Naive diversification on objective 1 (left) and objective 2 (right). Hatched areas correspond to the adjacent parts of the diversification subspace.

Large diversification

In general, the proximity assumption of the naive diversification is too strong. Particularly, small relaxations do not provide the search with enough freedom to find new diversified solutions. It is thus necessary to not constrain the search in a particular region of the objective space while ensuring that only nondominated solutions are discovered.

In large diversification, all the objective variables are configured in *No-Filtering* mode. Then, the **Pareto** constraint introduced in Section 4.1 is added to the problem. This constraint ensures that only solution that are not dominated by any solution in the archive can be discovered. The large diversification approach does not need to be driven by an objective as it is the case for the naive diversification approach. Observe that large diversification is less aggressive than naive diversification due to the larger search space to explore.

4.2.4 Impact of diversification and intensification

The benefits of including intensification along restarts is presented in Figure 4.5. The first setting only relies on diversification restarts. In the second setting, the rate of diversification/intensification is set to 50% for all the restarts.

One can see that after 5 seconds, the quality of the archive is clearly superior when using intensification restarts. The hypervolumes (averaged on 10 runs) are depicted on the right part of Figure 4.5. As expected, the hypervolume grows faster when including intensification restarts.

Notice that it may be useful to provide the intensification mode with more chance of improvement by relaxing a more important part of the solution when this optimization mode is selected.

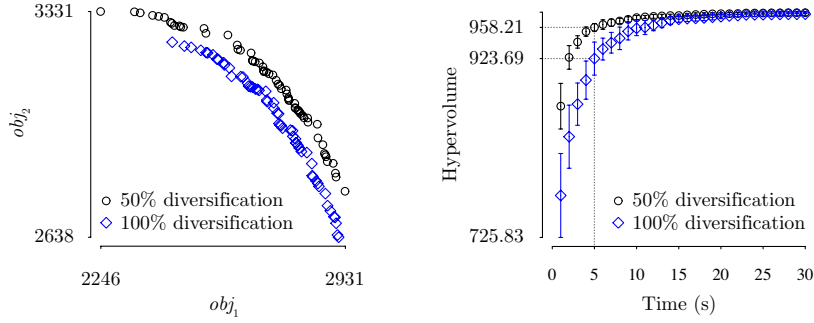


Figure 4.5: Impact of the diversification/intensification restart ratio on a 100 items bi-objective knapsack problem from MOCOLib [61]. (left) Nondominated solutions obtained after 5 seconds. (right) Evolution of the hypervolume.

4.3 Selection strategies

Before configuring the search in diversification mode or in intensification mode, MO-LNS has to select a solution in the archive. This solution is used to generate the neighborhood that will be explored with constraint programming. The selection of a solution is probably one of the main responsibilities of MO-LNS. Indeed, it is important to avoid spending time on solutions that cannot lead to an improvement of the archive.

The remainder of this section is dedicated to the presentation of different selection strategies.

4.3.1 Random selection

A simple way of selecting a solution is to choose it randomly in the archive. Despite what it might seem, random selection is not well suited for diversification. Indeed, solutions may be clustered in some particular parts of the objective space while some solutions may be isolated.² Hence, the random selection may spend a lot of restarts trying to improve the regions of the archive containing many solutions, while leaving small chances for regions with isolated solutions. This situation is illustrated in Figure 4.6.

4.3.2 Local Space Strategies

Initially designed for the naive diversification approach, *local space strategies* are based on the volume of the *local objective space* of each solution. The *local objective space* is defined as the region of the objective space delimited by the lower closest bounds l_j and the upper closest bounds u_j of its solution:

$$\forall i \in \{1, \dots, m\} : l_i \leq obj_i \leq u_i$$

²This is particularly true for academic problems – as the multi-objective traveling salesman problem or the multi-objective knapsack problem – where most of the solutions are situated in the curve of the Pareto front.

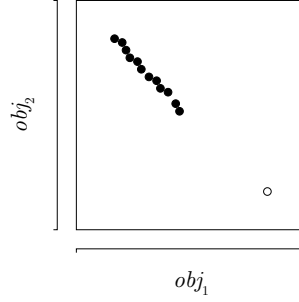


Figure 4.6: Some solutions of the archive may be clustered in some regions of the objective space (in black) while other solutions are isolated (white).

where the lower closest bound l_j of a solution sol is defined in a symmetric way as its upper closest bound u_j (see Equation 4.3):

$$l_j = \max(\{sol'(obj_j) \mid sol' \in \mathcal{A} \wedge sol'(obj_j) < sol(obj_j)\} \cup \{obj_j^{min}\}) \quad (4.4)$$

The idea behind *local space strategies* is to rank the solutions according to the volume of different regions of their *local objective space*.

Local Intensification Strategy. In this strategy, we are interested by the volume of the *local intensification space* that is the region of the *intensification subspaces* that intersects the *local objective subspace* of a solution (see Figure 4.7 left). The intuition of this strategy is that solutions with a large *local intensification subspace* are most likely to succeed in intensification. Hence, the *local intensification strategy* ranks the solutions according to the volume of their *local intensification subspace*.

Local Diversification Strategy. In this strategy, we are interested by the volume of the *local diversification space* that is the region of the *diversification subspaces* that intersects the *local objective subspace* of a solution (see Figure 4.7 right). The intuition of this strategy is that solutions with a large *local diversification subspace* are most likely to lead to new diversified solutions. Hence, the *local diversified strategy* ranks the solutions according to the volume of their *local diversification subspace*.

The *local diversification strategy* is well suited for diversification. Particularly, the solutions at the extremities of the archive may have very large *local diversification space*, leading to an important selection rate of these solutions (see Figure 4.8 left). This is an interesting behavior in the early iterations of MO-LNS that helps the archive to spread as much as possible. However, this selection strategy may get stuck once the extremities are well approximated. As well, nothing guarantees that the Pareto front is continuous on all its landscape (see Figure 4.8 right). Hence, it is interesting to add some randomness in the selection strategy to avoid spending restarts on unpromising solutions. For instance, one may select the solutions according to a probability distribution proportional to the volume of the *local diversification space* of each solutions.

Finally, local space strategies suffers from the computational cost needed to compute the closest upper bounds u_j and the lower closest bounds l_j of each

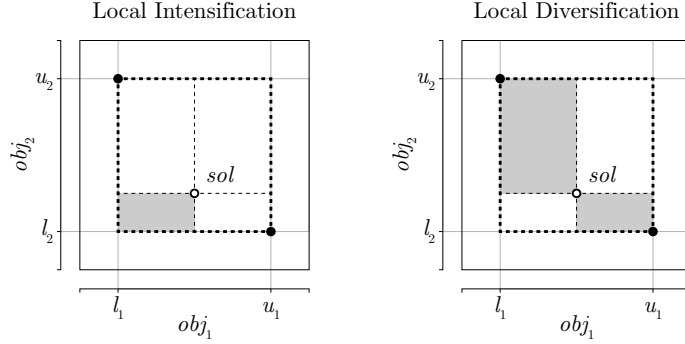


Figure 4.7: The *local objective space* of a solution (hatched square) is delimited by the lower closest bounds l_j and the upper closest bounds u_j of this solution. Gray areas correspond to the *local intensification space* (left) and the *local diversification space* (right).

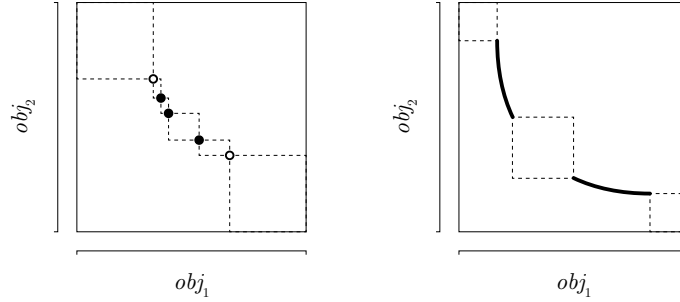


Figure 4.8: The landscape of the Pareto front may have a huge impact on the behavior of *local space strategies*. (left) The *local diversification strategy* prefers the extremities of the archive (white solutions) over all the other solutions. (right) Gaps in the landscape of the Pareto front may deceive the *local diversification strategy*.

solution.³

4.3.3 Nearest Neighbor Strategy

On the one hand, *random selection* might have negative side effects of reinforcing possible clusters in the archive. On the other hand, *local space strategies* have an important computational time and have to be tuned with some randomness or metaheuristics (as tabu [20]). Thus, we imagined another randomized strategy tending to fill-in the gaps more quickly.

The idea behind the *nearest neighbor strategy* is to select an uniform random point on the hyperplane formed by the extreme solutions of the archive (*i.e.* on a line for a bi-objective problem). Then, the solution that is the nearest (according to an Euclidean distance metric) one from this random point is selected. As shown in Figure 4.9, the nearest neighbor strategy tends to select the solutions near the gaps of the front. This strategy has a much lower computational cost as

³Some data structures (*e.g.* ordered linked-lists) allow to incrementally maintain these values.

the local space strategies since sampling on hyperplane is quite easy (particularly in bi-objective problem).

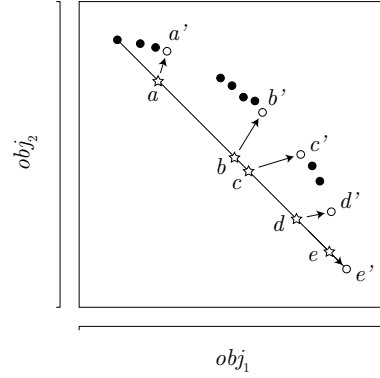


Figure 4.9: Selection of solutions according to the nearest neighbor strategy. The straight line represents the hyperplane defined by the extremities of the archive. The stars a , b , c , d , and e correspond to possible points randomly generated on the hyperplane. The solutions a' , b' , c' , d' , and e' are the solutions that would be selected for each of the random points.

4.3.4 Impact of the selection strategy

This section presents the performances of the different selection strategies presented in the previous section.

Table 4.1 compares the performances of Random Selection (RS), Local Diversification Strategy (LDS), and Nearest Neighbor Strategy (NNS) on four bi-objective knapsack problems from MOCOLib [61]. As we can see, random selection is always outperformed by both other strategies. The local diversification strategy and the nearest neighbor strategy seem to be equivalent in term of quality. However, the nearest neighbor strategy is always more than twice faster.

Figure 4.10 shows the benefits of the nearest neighbor strategy over random selection on a 200 items bi-objective knapsack problem from the MOCOLib [61]. We have initially added 6 Pareto optimal solutions in the archive. Then, 20 diversification restarts were executed with both strategies. While random selection (right) quickly focuses on a particular region of the objective space, the nearest neighbor strategy (left) diversifies the archive by trying to discover solutions between gaps.

Instance	Strategy	$ S $	$\mathcal{H}(S)$	Time(s)
2KP250A	RS	81.4	92.44	1.68
	LDS	153.1	93.35	7.87
	NNS	136.3	92.96	2.07
2KP250B	RS	63.7	87.75	1.36
	LDS	173.1	88.6	12.35
	NNS	149.1	88.73	2.02
2KP250C	RS	75.9	89.33	1.73
	LDS	126.3	90.07	5.99
	NNS	133.7	90.08	1.89
2KP250D	RS	64.4	65.60	1.31
	LDS	108.0	65.82	4.99
	NNS	100.2	65.82	1.68

Table 4.1: Comparison of different selection strategies on four bi-objective knapsack problems of 250 items. For each problem and each strategy (columns 1 and 2), we present the size of the archive (column 3), the hypervolume of the archive (column 4) and the time needed to perform 100 MO-LNS restarts (column 5). Results are averaged on 10 runs with different seeds.

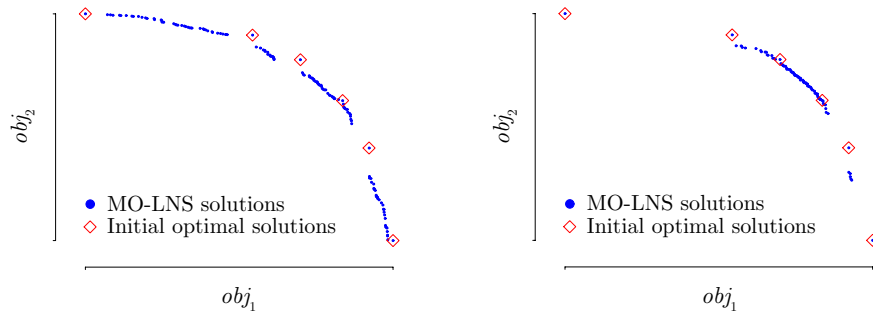


Figure 4.10: Impact of the selection strategy after 20 restarts on a 200 items bi-objective knapsack problem using 100% of diversification and starting from 6 initial Pareto optimal solutions. (left) nearest neighbor strategy. (right) random selection.

Chapter 5

Modeling Abstractions in OsaR

The main purpose of MO-LNS is to allow users to solve MOCO problems with the high-level language of constraint programming. Following this idea, we have developed a set of abstractions to model MOCO problems in a natural way. These abstractions are available in the main distribution of OsaR [38] *i.e.* an open-source CP solver developed in Scala [37].¹

This chapter is an introduction to modeling Multi-Objective Constraint Programming (MO-CP) problems. It presents the different abstractions and shows step-by-step how to model a multi-objective knapsack problem using OsaR (Section 5.1). The second part of this chapter explains how to extend an MO-CP model into an MO-LNS model (Section 5.2).

5.1 Multi-Objective Constraint Programming

Modeling MO-CP problems is really similar to modeling classical CP problems. A MO-CP model is composed of two main implementation blocks:

- First, users have to declare the decision *variables*² used in the problem definition (Section 5.1.1).
- Then, users have to state different *objectives* and *constraints* over the previously declared decisions variables (Section 5.1.2 and 5.1.3).

To solve an MO-CP problem, a third part has to be added to drive the exploration of the search tree induced by the model (Section 5.1.4). Since “searching” is quite orthogonal to “modeling”, MO-CP can be defined as the sum of a model and a search procedure:

$$\text{MO-CP} = \text{Model} + \text{Search}$$

The generic framework of MO-CP is presented in Statement 5.1 where `cp` is the CP solver responsible for solving the problem.

¹We assume that the reader is familiar with the Scala programming language.

²To avoid confusion, “variables” will always refer to the variables of the problem, not variables `var` in Scala.

```

1  val cp = CPSolver()
2  // Declaration of decision variables
3  cp.paretoMinimize(obj1, ..., objm) subjectTo {
4    // Post the Constraints
5  } exploration {
6    // Search Heuristic
7  } run

```

Statement 5.1: Framework of an MO-CP problem.

5.1.1 Variables

As mentioned above, the first step in modeling a problem is to declare its variables. In *Oscar*, two types of variables are available:

- Discrete integer variables: `CPVarInt`,
- Boolean variables: `CPVarBool`.

Discrete integer variables are the most commonly used and are sufficient to model a large scope of problems. A `CPVarInt` must be given a `CPSolver` and a domain to be instantiated. For example, we can declare a variable `intVar1` with a domain `[1..10]` using the following statement:

```
val intVar1 = CPVarInt(cp, 1 to 10)
```

We can also define the domain of a `CPVarInt` using a set of integers `Set[Int]`. The following statement declares a new variable `intVar2` with domain `{2, 5, 9}`:

```
val intVar2 = CPVarInt(cp, Set(2, 5, 9))
```

In *Oscar*, boolean variables are discrete integer variables with domain `{1, 0}` where 1 corresponds to `true` and 0 to `false`. Therefore, `CPVarBool` variables only require a `CPSolver` as parameter:

```
val boolVar = CPVarBool(cp)
```

5.1.2 Objectives

Multi-objective problems require to minimize or maximize a set of objective. In *Oscar*, an objective is defined as a `CPVarInt`. The minimal and maximal values in the domain of an objective represent respectively its lower bound and its upper bound. For example, the following statement declares two objectives such that $\text{obj1} \in [1..15]$ and $\text{obj2} \in [5..10]$:

```
val obj1 = CPVarInt(cp, 1 to 15)
val obj2 = CPVarInt(cp, 5 to 10)
```

The methods `paretoMinimize` and `paretoMaximize` are used to tell the CP solver which are the objectives to minimize or to maximize. The objectives have to be passed as a sequence of `CPVarInt`. In the following instruction, the CP

solver `cp` is tell to search for all the optimal solutions minimizing the value of `obj1`, `obj2`, and `obj3`:

```
cp.paretoMinimize(obj1, obj2, obj3)
```

More control on which objectives have to be maximized or minimized can be obtained. The method `paretoOptimize` takes a sequence of tuple (`CPVarInt`, `Boolean`) where `CPVarInt` represents an objective and the `Boolean` is set to `true` (resp. `false`) if the objective has to be maximized (resp. minimized). The following instruction tells the CP solver to maximize `obj1` and `obj2` while minimizing `obj3`:

```
cp.paretoOptimize((obj1, true), (obj2, true), (obj3, false))
```

5.1.3 Constraints

The constraints of a problem have to be stated in the `subjectTo` part of the model. OscaR provides users with a large set of constraints to model problems naturally and efficiently. An exhaustive list of these constraints is available in [38].

Each constraint is an extension of the class `Constraint`. To be considered, a constraint has to be posted in `CPSolver` with the methods `add` or `post`. The difference between, `add` and `post` is that `add` will throw an exception if adding the constraint turns the problem unsatisfiable.³ Notice that the propagation algorithm is automatically called after posting a constraint (see Section 1.3.2).

The next statement enforces two constraints. The first, constrains `x` to be lower than `y` while the second constrains the sum of `x` and `y` to be lower than 10. Posting the constraint `x < y` and `x + y < 10` calls the propagation algorithm and strengthens the domains of `x` and `y` to `[1..7]` and `[2..8]`.

```
val x = CPVarInt(cp, 1 to 10)
val y = CPVarInt(cp, 1 to 10)

cp.subjectTo {
  cp.post(x < y)
  cp.add(x + y < 10) // May throw an exception if unfeasible
}
```

Actually, the constraint `x + y < 10` is decomposed into two different constraints:

- The first is a `sum` constraint enforcing an implicit variable `z` to be equal to the sum of `x` and `y`;
- The second is a `lessThan` constraint enforcing the value of `z` to be less than 10.

This kind of implicit abstractions are common in OscaR and helps the user to state constraints as easily as possible. For example, the `Pareto` constraint (see

³The method `add` is particularly useful to detect bugs when stating the constraints of the problem.

Section 4.1) is automatically configured and posted when stating the objectives using the methods `paretoMinimize`, `paretoMaximize`, or `paretoOptimize`.

5.1.4 Exploration

Finding a solution to an MO-CP problem requires a search procedure to explore the search space as a search tree. In OscaR, the function `branch` can be used to define a binary search tree by considering two alternatives (see Section 1.4.2). In general, the first alternative (*i.e.* left branch) consists in assigning a value `v` to a selected variable `x` by posting the constraint `x == v`. The second alternative (*i.e.* right branch) consists in forbidding the assignment of the value `v` to the variable `x` by posting the constraint `x != v`:

```
cp.exploration {
  cp.branch { cp.post(x == v) }{ cp.post(x != v) }
}
```

OscaR provides users with predefined non-deterministic generic search procedures [56].⁴ Nevertheless, users are encouraged to define their own, problem specific, ways of exploring the search tree. For example, let us consider the following statement in which the search procedure selects the variable in `vars` (*i.e.* a vector of variables) with the smallest domain and tries to assign it to its smallest value.

```
cp.exploration {
  while (!allBounds(vars)) {
    val x = selectMin(vars)(!_.isBound)(_.size).get
    cp.branch { cp.post(x == x.min) }{ cp.post(x != x.min) }
  }
}
```

The function `allBounds` returns `false` if at least one variable in `vars` is unbound *i.e.* the domain of the variable contains more than one value. Conversely, `allBounds` returns `true` when the assignment on `vars` is complete.

The function `selectMin` is really convenient for designing search heuristics. It returns the minimal element of a sequence according to a filtering function and an ordering function. In the previous statement, `selectMin` is used to select the unbound variable with the smallest domain.

5.1.5 Starting the Search

At this point, we have seen how to define the model and the search of an MO-CP problem. Therefore, all it remains is to ask the CP solver to find a solution to the problem or to prove its unfeasibility otherwise.

The `run` method of `CPSolver` is used to start exploring the search tree. By default, the search will stop when the entire search tree has been explored.⁵ Hence, it is necessary to fix some limits on the exploration to avoid waiting for a solution for ages. In OscaR, tree types of limit are available:

⁴A search is said non-deterministic when the topology of the search tree is determined during the exploration.

⁵We say that the search is exhausted.

- It is possible to set a limit on the number of encountered feasible solutions. For example `run(nbSolMax = 1)` will stop the search when a first feasible solution is found. However, the search will continue until proving the unsatisfiability of the problem if no solution exists.
- To avoid this issue, the search can be limited by a maximal number of allowed backtracks. The `failureLimit` parameter of `run` is used to fix this limit.
- Finally, the `timeLimit` parameter can be passed to `run` to limit the execution time of the search.

Notice that limits can be combined to obtain more specific stop criterion. For example, the following instruction asks for the first three solutions encountered within a limit of 2000 backtracks:

```
cp.run(nbSolMax = 3, failureLimit = 2000)
```

5.1.6 Solutions and Archive

Since solving a problem without recording its solutions does not make much sense, OscaR provides users with `CPSol` objects to register solutions. A `CPSol` object is a dictionary that maps each decision variable to its assigned value in the corresponding solution. To build a solution object, the CP solver needs to know which variables are decision variables and which are not. The method `addDecisionVariables` of `CPSolver` was designed to do so. For example, the next instructions ask to register the variables `x` and `y` as decision variables:

```
val x = CPVarInt(cp, 1 to 10)
val y = CPVarInt(cp, 1 to 10)
cp.addDecisionVariables(x, y)
```

As shown, `addDecisionVariables` takes a sequence of variables as parameter. As well, it is possible to pass directly a vector of variables to the method.

```
val vars = for (i <- 1 to 10) yield CPVarInt(cp, 1 to 10)
cp.addDecisionVariables(vars)
```

The objectives stated using the methods `paretoMinimize`, `paretoMaximize`, or `paretoOptimize` are automatically considered as decision variables.

Each time a solution is encountered during the search, a `CPSol` is instantiated mapping each decision variable to its assigned value. Then, this new solution is inserted into the archive maintained by `CPSolver`. The solutions in the archive can be accessed using the `nonDominatedSols` function of `CPSolver`

```
val sols = cp.nonDominatedSols
```

Notice that this archive is the one used by the `Pareto` constraint to filter dominated branches of the search tree (Section 4.1).⁶

⁶The archive is implemented as a linear-list (see Section 7.1).

5.1.7 Example: the Multi-Objective Knapsack Problem

This section describes how to use OscaR to model the Multi-Objective Knapsack Problem (MOKP) defined in Section 2.4.1.

Before starting modeling the MOKP, let us introduce the data of the problem in the following lines. Lines 1 and 2 declare the values `nObjs` and `nItems` and should be self-explanatory. Then, lines 3 and 4 declare `profit` and `weight`, two matrices of size `nObjs` $\times$ `nItems` which contain the profit and the weight of each item according to each objective. The maximal capacity of each knapsack is stored in `capa` (line 5). Finally, `Objs` and `Items` are range over the items and the objectives (lines 6 and 7).

```
1  val nObjs: Int           // Number of objectives
2  val nItems: Int          // Number of items
3  val profit: Array[Array[Int]] // Matrix nObjs x nItems of profits
4  val weight: Array[Array[Int]] // Matrix nObjs x nItems of weights
5  val capa:  Array[Int]      // Maximal capacities
6  val Objs  = 0 until nObjs  // Range of objectives
7  val Items = 0 until nItems // Range of items
```

Variables

The decision variables of the MOKP are contained in a vector `x` of boolean variables such that a variable `x(i)` is equal to 1 if the item `i` is packed in all the knapsacks; it is equal to 0 otherwise. The vector `x` can be declared as follows:

```
val x = for (i <- Items) yield CVarBool(cp)
```

It is also necessary to introduce a vector of auxiliary variables representing the actual charge of each knapsack. To do so, we can declare a vector of integer variables such that the domain of each variable contains all the possible charges for the knapsacks:

```
val capaVar = for (o <- Objs) yield CVarInt(cp, 0 to capa(o))
```

Interestingly, using the maximal capacity as an upper bound for the variables in `capaVar`, induces the constraint of not exceeding the maximal capacity of each knapsack.

Objectives

The objectives to maximize are the total profits of each knapsack. Again, these variables can be modeled using a vector of integer variables `profitVar`:

```
val profitVar = for (o <- Objs) yield CVarInt(cp, 0 to profit(o).sum)
cp.paretoMaximize(profitVar)
```

Observe that the domain of each objective is built using two extremely bad approximations of the lower and upper bounds. Nevertheless, this is not really a problem in this case. Indeed, the domain of these objectives will be automatically reduced by the constraint of the problem, giving better approximations of

the bounds.⁷

Constraints

Since the capacity constraint of each knapsack is enforced by the domain of its charge in `capaVar`, the only constraints used to model the problem are *channeling* constraints making the link between items, profits, and charges. We can model these constraints using the `sum` constraint:

```
for (o <- Objs) {
  cp.add(capaVar(o) == sum(for (i <- Items) yield x(i)*weight(o)(i)))
  cp.add(profitVar(o) == sum(for (i <- Items) yield x(i)*profit(o)(i)))
}
```

However, it is more appropriate to use the constraint `binaryKnapsack` [49] implemented in `OscAR`. This constraint has the main advantage of taking the combinatorial structure of the problem in consideration to improve the quality of its filtering. The constraints of the multi-objective knapsack problems become the followings:

```
for (o <- Objs) {
  cp.add(binaryKnapsack(x, profit(o), weight(o), profitVar(o), capaVar(o)))
}
```

Search

The heuristic used to explore the search tree of the multi-objective knapsack problem is based on a greedy algorithm. The idea is that items with a low ratio weight over profit will have a better chance to be part of an optimal solution. This heuristic is strongly related to single-objective problems as we need to aggregate the weights and profits to compute the ratio of an item:

$$ratio_i = \frac{\sum_j^m \lambda_j w_{ij}}{\sum_j^m \lambda_j p_{ij}}, \quad \sum_j^m \lambda_j = 1.$$

This is not really a problem since we are interested in finding good solutions as soon as possible to improve the pruning of the `Pareto` constraint.

In the following search procedure, the Scala variable `obj` is used to drive the search towards good solutions according to the objective `obj`:

```
var obj = 0
while (!allBounds(x)) {
  val item = selectMin(Items)(!x(_).isBound) {
    i => weight(obj)(i) / profit(obj)(i)
  }.get
  cp.branch{cp.post(x(item) == 1)}{cp.post(x(item) == 0)}
}
```

⁷This is only true if the constraints do not make holes in the domain of the variables (see Definition 1.10).

The function `selectMin` returns the unassigned item with the lower ratio weight over profit. Then, `branch` is used to consider both alternatives: either the item is assigned to all the knapsack (left branch) or it is discarded (right branch).

5.2 Multi-Objective Large Neighborhood Search

One advantage of MO-LNS lies in its simplicity of implementation. An MO-CP model can be extended into an MO-LNS one by adding four implementation blocks:

- Select the next solution to relax. A basic selection procedure should be to select the next solution randomly in the archive.
- Select an objective to optimize. This objective can be selected randomly or using more sophisticated heuristic. It is also possible to select iteratively all the objective after selecting a solution from the archive.
- Select the optimization mode. Full diversification already provides much better results than pure MO-CP model. Intensification should be use to make the search more aggressive. Selecting the next mode may be done randomly according to a probability distribution.
- Relax the selected solution. Generic relaxation procedures can be used to relax the solution. However, it is recommended to use a specific relaxation according to the selected objective. Notice that the exploration heuristic should also be influenced by the selected objective.

The framework of an MO-LNS procedure is presented in Statement 5.2. Observe that it is not necessary to modify the MO-CP model to turn it into an MO-LNS one.

```

8  while(stopCriterion) {
9      // Select an objective to drive the iteration
10     // Select a solution to restart from
11     // Select an optimization mode
12     cp.runSubjectTo {
13         // Post the selected fragments of the solution
14     }
15 }
```

Statement 5.2: Framework of a MO-LNS problem.

5.2.1 Solution Selection

As mentioned in Section 5.1.6, the solutions can be accessed using the function `nonDominatedSols` of CPSolver. Therefore, users should use this vector of solutions to select a solution. For example, a random selection can be easily implemented as follows:

```

val rand = Random // Generator of random numbers
val sols = cp.nonDominatedSol
val sol = sols(rand.nextInt(sols.size))

```

Nevertheless, more sophisticated selection procedures as the nearest neighbor strategy (see Section 4.3.3) could be used.

5.2.2 Intensification and Diversification

The intensification or diversification mode can be stated really easily. For example, the following instruction configures the objectives for diversification:

```
cp.objective.diversify
```

The intensification mode requires the selected solution to adjust the bound of each objective. In case of driven intensification, it is also necessary to provide the selected objective:

```

cp.objective.intensify(sol) // Strong intensification
cp.objective.intensify(sol, obj) // Driven intensification

```

Selecting the optimization mode can be easily done randomly according to some probability distributions. In the following statement, `prob` is the probability, in percent, of selecting the diversification mode.

```

val prob = 50 // Probability of diversification
if (rand.nextInt(100) < 50) cp.objective.diversification()
else cp.objective.intensification(sol, obj)

```

5.2.3 Relaxation

The relaxation procedure aims at maintaining some fragments of a solution to generate a neighborhood to explore. In general, these fragments are maintained by adding temporary additional constraints to the problem. Then, a search is started to explore the neighborhood induced by these constraints.

Since temporary constraints have to disappear when the search terminates, they cannot be stated as usual constraints in the `subjectTo` part of the model. In OscaR, temporary constraints have to be stated inside the method `runSubjectTo`. For example, the following statement starts a new search with the temporary constraint `x = 1`:

```

cp.runSubjectTo(failureLimit = 500) {
  cp.add(x == 1)
}

```

As shown, `runSubjectTo` can take the same parameters as `run` to limit the time spent exploring the generated neighborhood. Notice that OscaR provides users with an easy way to relax random part of the problem with the `relaxRandomly` method.

5.2.4 Search

One particularity of MO-LNS is that it allows users to reuse single-objective relaxations and search procedures into a multi-objective context. Search procedures can be generic or problem specific. In this second case, it is important to drive the exploration w.r.t. the selected objective. This can be done by designing objective specific search procedures or by adapting the variable/value heuristic.

5.2.5 Example: the Multi-Objective Knapsack Problem

The MO-LNS extension of the MO-CP model presented in Section 5.1.7 gathers up the generic procedures developed above. The whole model of the MO-LNS multi-objective knapsack problem is presented in Statement 5.3 with the following implementation choices:

- The solution to relax is randomly selected in the archive (lines 35 and 36);
- The objective is randomly selected and modify the value of the Scala variable `obj` (line 38);
- The optimization mode is selected randomly with equal probability (lines 40 and 41);
- The neighborhood to explore is generated by relaxing randomly `relaxSize` decision variables of the selected solution (lines 43 and 44).

```

1  val nObjs : Int           // Number of objectives
2  val nItems : Int          // Number of items
3  val profit : Array[Array[Int]] // Matrix of profits
4  val weight : Array[Array[Int]] // Matrix of weights
5  val capa : Array[Int]      // Maximal capacities
6  val Items = 0 until nItems // Range of Items
7  val Objs = 0 until nObjs   // Range of objectives
8
9  val cp = CPSolver()
10 var obj = 0 // Drive the search
11
12 val x = for (i <- Items) yield CPVarBool(cp)
13 val capaVar = for (o <- Objs) yield CPVarInt(cp, 0 to capa(o))
14 val profitVar = for (o <- Objs) yield CPVarInt(cp, 0 to profit(o).sum)
15
16 cp.paretoMaximize(profitVar) subjectTo {
17   for (o <- Objs) {
18     cp.add(binaryKnapsack(x, profit(o), weight(o), profitVar(o), capaVar(o)))
19   }
20 } exploration {
21   while (!allBounds(x)) {
22     val item = selectMin(Items){!x(_).isBound} {
23       i => weight(obj)(i) / profit(obj)(i)
24     }.get
25     cp.branch { cp.post(x(item) == 1) } { cp.post(x(item) == 0) }
26   }
27 } run(nbSolMax = 1)
28
29 val nRelax : Int // Size of the relaxation
30 val maxIter : Int // Stop criterion
31 val limit : Int // Search limit
32 val rand = Random // Generator of random numbers
33
34 for (iter <- 1 to maxIter) {
35   val sols = cp.nonDominatedSolutions
36   val sol = sols(rand.nextInt(sols.size))
37
38   obj = rand.nextInt(nObjs)
39
40   if (rand.nextBoolean) cp.objective.diversify
41   else cp.objective.intensify(sol, obj) // Driven intensification
42
43   cp.runSubjectTo(failureLimit = limit) {
44     relaxRandomly(x, sol, relaxSize)
45   }
46 }

```

Statement 5.3: MO-LNS model for the Multi-Objective Knapsack.

Chapter 6

Experimental Results

During the last decades, many methods were proposed to tackle multi-objective combinatorial optimization problems [15]. Many of these methods are designed to solve standard academic problems such as the multi-objective knapsack problem or the multi-objective quadratic assignment problem. Particularly, hybridized techniques between Evolutionary Algorithms (EA) and Local Search (LS) offers impressive results on those problems and are able to provide users with good approximations of the exact Pareto front within a few seconds [11, 33, 63].

MO-LNS is a framework designed to provide users with good approximation of the exact Pareto front of large scale complex multi-objective problems. Therefore, the goal of MO-LNS is not to compete against state-of-the-art methods on standard academic problems. Indeed, these problems are by nature not well suited for constraint programming due to their simple combinatorial structure and to their small number of constraints. Still, academic problems are interesting benchmarks to study, with known exact Pareto front. Hence, we choose to assert the performance of MO-LNS on two of these problems. The third benchmark we chose is a large complex problem that is really difficult to tackle with current multi-objective approaches. This problem is more constrained and therefore more suited for constraint programming.

This chapter presents the experimental results obtained on the multi-objective quadratic assignment problem (Section 6.1), the multi-objective knapsack problem (Section 6.2), and the bi-objective tank allocation problem (Section 6.3).¹ All experiments were conducted with the OsaR open-source solver [38] on an Intel® Core i7™ 2.6GHz CPU.

6.1 Multi-Objective Quadratic Assignment

We experiment MO-LNS on the Multi-Objective Quadratic Assignment Problem (MOQAP) introduced in Section 2.4.2. The benchmark is composed of instances of 10 facilities (from [30]), for which the exact Pareto fronts are known.

¹Instances and exact Pareto fronts are available at <http://becool.info.ucl.ac.be/resources/mo-lns>

Instance	$ S^* $	$\mathcal{H}(S^*)$	Algorithm	$\mathcal{H}(S)$	$ S $	$ S \cap S^* $
KC10-2fl-1uni	13	117.38	MO-LNS	115.43	9	6.6
			MO-CP	99.09	15	0
KC10-2fl-2uni	1	91.56	MO-LNS	87.11	1.4	0.6
			MO-CP	76.99	2	0
KC10-2fl-3uni	130	90.5	MO-LNS	87.42	84.4	30.6
			MO-CP	78.79	65	0
KC10-2fl-1rl	58	606005.79	MO-LNS	604932.58	54	50.4
			MO-CP	598146.6	41	13
KC10-2fl-2rl	15	604864.4	MO-LNS	604864.4	15	15
			MO-CP	604864.4	15	15
KC10-2fl-3rl	55	623898.5	MO-LNS	623076.5	47	43
			MO-CP	565772.58	37	0
KC10-2fl-4rl	53	732716.05	MO-LNS	732716.05	53	53
			MO-CP	732716.05	53	53
KC10-2fl-5rl	49	1819669.88	MO-LNS	1819669.88	49	49
			MO-CP	1819669.88	49	49

Table 6.1: Comparison of MO-CP and MO-LNS on standard instances of the MO-QAP. MO-LNS always provides equivalent or better results than MO-CP.

6.1.1 MO-LNS settings

The parameters selected to solve the multi-objective quadratic assignment problem are quite standard. The next solution to relax is selected using the *nearest neighbor strategy*. At each iteration, the diversification mode is selected with a probability of 50%. The relaxation procedure is not objective driven and consists in relaxing randomly 50% of the decisions variables.

The used variable/value heuristic attempts to minimize the largest terms of the objective function on which it is still possible to act. Those large terms are induced by assignments of facilities i and j with a large weight w_{ij} and placed on distant locations. As well as the relaxation procedure, this search heuristic is not objective driven. Each restart is given a limit of 500 backtracks.

6.1.2 Results

Table 6.1 reports the results obtained with a 30 seconds time-out, averaged over 10 runs for MO-LNS. The size and the hypervolume of the exact Pareto fronts are given in columns 2 and 3. The hypervolumes obtained with MO-LNS and MO-CP are given in column 4. Then, the sizes of the archives obtained with MO-LNS and MO-CP ($|S|$) are given in column 5. Finally, the number of optimal solutions obtained with each approach ($|S \cap S^*|$) is presented in column 6. Notice that all the following tables respect this columns configuration.

As can be seen, MO-CP solves 3 instances to optimality. MO-LNS is also able to solve these instances to optimality and obtain strictly better results on

the remaining instances. The hypervolume values reached by MO-LNS are very close to the optimal ones.

6.2 Multi-Objective 0/1 Knapsack

The multi-objective 0/1 knapsack problem is a specific instance of the general multi-objective knapsack problem introduced in Section 2.4.1. In this problem, all the knapsacks $j \in \{1, \dots, m\}$ have the same maximal capacity C . As well, each item $i \in \{1, \dots, n\}$ has the same weights w_{ij} regardless of the knapsack j . Therefore, the multi-objective 0/1 knapsack problem consists in packing items with different profits into a unique knapsack while trying to maximize the sum of each profits.

6.2.1 MO-LNS settings

The next solution *sol* to restart from is chosen in the archive with the nearest neighbor strategy. The idea of the relaxation procedure is to keep fixed some *good items* (w.r.t. the selected objective) already assigned in *sol* :

1. select randomly one objective index $j \in \{1, \dots, m\}$,
2. select randomly 90% of the items in the set $\{i \in \{1, \dots, m\} \mid \text{sol}(x_i) = 1\}$ according to a probability function proportional to p_{ij}/w_i ,
3. keep fixed the items selected at step 2 *i.e.* force them to be in the knapsack,
4. chose randomly to diversify or intensify with equal probability.

The variable/value heuristic used to explore the neighborhood generated by relaxing the selected solution is dependent of the selected objective j . The heuristic first selects the unbound variable x_i with the largest ratio p_{ij}/w_i , assigning the corresponding item on the left branch and discarding it on the right branch. Each restart is given a limit of 1000 backtracks.

6.2.2 Results

We compare the MO-CP and MO-LNS approaches on instances from the MO-COLib [61] ranging from 50 to 250 items. Table 6.2 reports the results obtained with a 60 seconds time-out, averaged over 10 runs for MO-LNS.²

As can be seen, MO-LNS consistently obtains better or equivalent results compared to the MO-CP approach. The hypervolume values reached by MO-LNS are very close to the optimal ones.

6.3 Bi-Objective Tank Allocation

As presented in Section 2.4.3, the tank allocation problem consists in assigning different cargoes to the available tanks of a vessel while satisfying hard segregation constraints *e.g.* to avoid placing dangerous cargoes in adjacent tanks. The first instance – chemicalA – is based on a real world configuration of the

²Table 6.2 has the same columns configuration as Table 6.1.

Instance	$ S^* $	$H_{S^*}(10^6)$	Algorithm	$H_S(10^6)$	$ S $	$ S \cap S^* $
2KP100A	172	15.59	MO-LNS	15.59	172	172
			MO-CP	15.05	128	112
2KP100B	174	15.12	MO-LNS	15.12	170.60	164.70
			MO-CP	14.62	124	93
2KP100C	64	16.68	MO-LNS	16.68	64	64
			MO-CP	16.68	64	64
2KP100D	76	16.31	MO-LNS	16.31	76	76
			MO-CP	16.28	73	73
2KP150A	244	39.66	MO-LNS	39.66	226.20	187
			MO-CP	35.42	88	31
2KP150B	348	41.46	MO-LNS	41.46	303.30	192.40
			MO-CP	36.31	91	51
2KP150C	166	34.17	MO-LNS	34.17	155.60	127
			MO-CP	33.15	83	34
2KP150D	207	36.04	MO-LNS	36.04	199.80	155.60
			MO-CP	32.88	88	62
2KP200A	439	64.34	MO-LNS	64.34	361	178.20
			MO-CP	57.43	86	34
2KP200B	397	65.78	MO-LNS	65.77	345.20	232.80
			MO-CP	58.82	108	54
2KP200C	328	57.48	MO-LNS	57.46	297.80	187.10
			MO-CP	48.30	60	18
2KP200D	361	73.42	MO-LNS	73.40	304.10	176.90
			MO-CP	62.71	64	29
2KP250A	629	94.37	MO-LNS	94.34	433.50	95.50
			MO-CP	78.68	70	12
2KP250B	629	89.67	MO-LNS	89.65	410.90	107.90
			MO-CP	74.81	90	44
2KP250C	528	91.25	MO-LNS	91.24	383.30	108.90
			MO-CP	75.30	72	22
2KP250D	424	66.56	MO-LNS	66.55	303.40	142.80
			MO-CP	56.98	51	26

Table 6.2: Comparison of MO-CP and MO-LNS on standard instances of the Bi-Objective Knapsack Problem. MO-LNS always provides equivalent or better results than MO-CP alone.

Instance	$ S^* $	$\mathcal{H}(S^*)$	Algorithm	$\mathcal{H}(S)$	$ S $	$ S \cap S^* $
chemicalA	6	1848	MO-LNS	1848	6	6
			MO-CP	1022	4	1
chemicalB	7	2976	MO-LNS	2976	7	7
			MO-CP	1010	3	0
chemicalC	8	3597	MO-LNS	3597	8	8
			MO-CP	852	2	0
chemicalD	8	5358	MO-LNS	5358	8	8
			MO-CP	555	1	0

Table 6.3: Comparison of MO-CP and MO-LNS on four instances of the bi-objective tank allocation problem. MO-LNS is able to find the exact Pareto front of each instance within a 60 seconds time-out.

problem. Instances chemicalB, chemicalC, and chemicalD have been generated from chemicalA by modifying the volume of each cargo. The LNS model used in our experiment is the same as the one introduced in [48] except that it is now bi-criteria.

6.3.1 MO-LNS settings

The MO-LNS parameters are as follows:

- select the next solution to relax using *random selection*.³
- select randomly one objective index $j \in \{1, \dots, m\}$,
- chose randomly to diversify or intensify with equal probability.

The relaxation procedure and the search heuristic are driven by the selected objective and are the same as the ones introduced in [48]. Each restart is given a limit of 2000 backtracks.

6.3.2 Results

Results are presented in Table 6.3. The exact Pareto fronts were generated with the van Wassenhove and Gelders algorithm (see Section 3.1) using a MIP solver (Gurobi 5.02 [22]) that took about 3 minutes to run on each instance. The MO-LNS framework finds the exact Pareto front of each instance within a time-out of 60 seconds. We also indicate for the same model the results obtained for a MO-CP approach with a time-out of 300 seconds. This MO-CP approach is only able to discover one solution of the exact Pareto front of the first instance (chemicalA). The hypervolume indicator shows that other nondominated solutions discovered with MO-CP remain quite far from optimal ones.

³The selection strategy does not have a noteworthy impact on this problem since the Pareto fronts are quite small.

Figure 6.1 presents the archives generated by MO-LNS and MO-CP on the first instance *e.g.* chemicalA. Only 303 MO-LNS restarts and 33 seconds are necessary to discover the 6 Pareto optimal solutions of the problem. Particularly, MO-LNS is able to find an archive really close to the exact Pareto front within a time-out of 10 seconds.

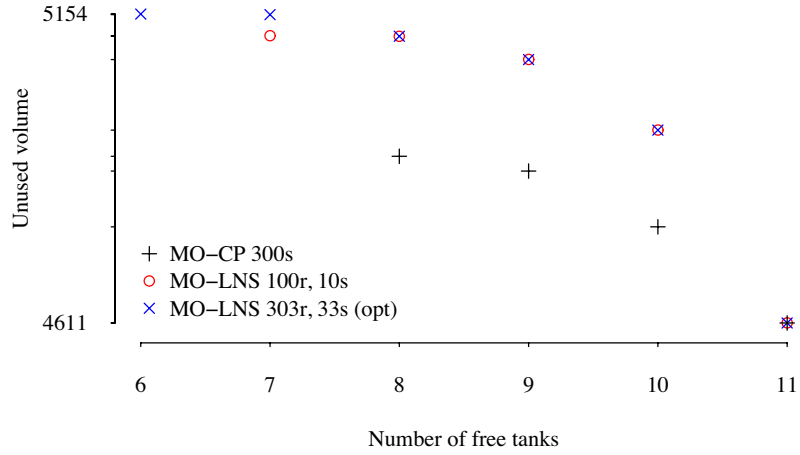


Figure 6.1: Comparaision of MO-CP and MO-LNS on the chemicalA instance of the bi-objective tank allocation problem. MO-LNS is able to find the exact Pareto front in 33 seconds. MO-LNS is already able to find a good approximation of the exact Pareto font within a 10 seconds time-out.

Chapter 7

Data Structures

The *archive* is certainly one of the most critical component of many multi-objective algorithms. Particularly, the filtering algorithm of the **Pareto** constraint (section 4.1) makes intensive use of the archive. Therefore, the choice of a specific data structure to implement the archive has a strong influence on the global efficiency of many multi-objective methods.

Basically, the insertion of a new candidate solution is the main operation available on an archive. As presented in Definition 2.10, an archive $\mathcal{A}$ has to hold the following *domination-free* property:

$$\forall sol \in \mathcal{A}, \nexists sol' \in \mathcal{A} : sol' \prec sol$$

The insertion operation thus has to handle the inserted solutions to respect this property. We distinguish three execution cases when inserting a candidate solution in the archive:

1. The candidate solution is dominated by a solution in the archive. The archive is not affected by the operation and the candidate solution is discarded;
2. The candidate solution is not dominated and does not dominate any solution in the archive. The candidate solution is inserted into the archive without removing any other solution;
3. The candidate solution dominates some solutions in the archive. All the dominated solutions are removed and the candidate is inserted into the archive. We say that the candidate solution *cleans* the archive.

In this chapter, we describe different data structures commonly used to implement an archive. First, we present the linear-list data structure which is probably the simplest way to implement an archive (Section 7.1). Then, we focus on the efficient implementation of bi-objective archives based on an extension of standard balanced binary trees (Section 7.2). Finally, we describe the domination-free quad-tree data structure and we explain how to improve this data structure with a new algorithm to *clean* the archive (Section 7.3).

7.1 Linear-list

As mentioned above, using a linear-list is probably the most straightforward way to implement an archive. No assumption is made on the ordering of the solutions in the data structure. Thus, it is necessary to compare the candidate solution to all the solutions in the linear-list to ensure that it is not dominated or to remove all the solutions that are dominated by this candidate. Clearly, the time complexity of the insertion operation is $\mathcal{O}(n)$ where n is the size of the archive. However, the algorithm may terminate faster if a dominating solution is encountered early in the data structure.

7.2 Balanced Linked-Tree

As mentioned in Section 3.1, bi-objective problems have the particularity that improving the first objective of a Pareto optimal solution cannot be done without degrading the value of the second objective. Hence, sorting the solutions of a bi-objective problem in increasing order w.r.t. one objective amounts to sorting these solutions in decreasing order w.r.t. the other objective. Furthermore, if one knows the position of the candidate solution in the ordered list of solutions, it is possible to test in a constant time if the candidate dominates or is dominated by some solutions in the archive.

Balanced binary trees as AVL trees [2] or red-black trees [4] are well known data structures that ensure the access of their elements in a worst time complexity of $\mathcal{O}(\log n)$. Balanced linked-trees (or braided balanced trees [43]) are not as well known as standard balanced trees. A balanced linked-tree is an ordered linked-list and a balanced binary tree at a same time. Particularly, balanced linked-trees ensure a worst time complexity of $\mathcal{O}(\log n)$ for basic operations as *access*, *insertion* and *deletion*. An arbitrary balanced linked-tree is illustrated in Figure 7.1.

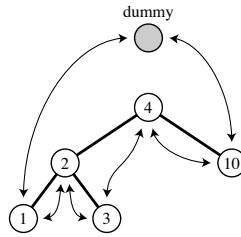


Figure 7.1: An arbitrary balanced linked-tree. Arrows correspond to the successor/predecessor pointers. The dummy node (in gray) is used for ease of implementation. Its successor and its predecessor are the first and the last elements of the ordered linked-list respectively.

Balanced linked-trees seem to be good candidates for storing the archive of bi-objective problems. Without focusing on the implementation details, this section presents a way of extending standard balanced binary trees to maintain an ordered linked-list. Then, a new¹ algorithm for domination-free insertion is presented.

¹At the best of our knowledge, this algorithm has never been presented in a scientific publication.

7.2.1 Maintaining the linked-list

Basically, the insertion and deletion are the only operations to update the structure of an ordered linked-list. Therefore, we need to extend the insertion and deletion operations of the balanced binary tree to maintain the ordered linked-list. Observe that the rebalancing operations (known as rotations) of balanced binary trees do not affect the elements contained in the ordered linked-list and do not need to be extended (see Figure 7.2).

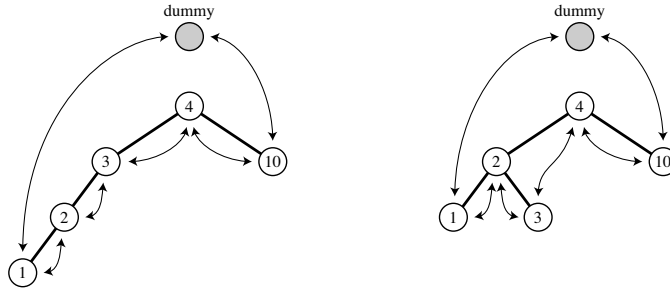


Figure 7.2: A balanced binary linked-tree (left) before rotations and (right) after rotations. The structure of the ordered linked-list is not modified by the rotations.

As mentioned above, the ordered linked-list is updated each time a new element is inserted in the balanced linked-tree. For ease of implementation, we add a *dummy* node to the structure of the ordered linked-list. The predecessor and the successor of the dummy node correspond to the first and last elements of the ordered linked-list respectively. When the tree is empty, the dummy node is its own predecessor and successor (see Figure 7.3a). Let *new* denote the node containing the new inserted element, the insertion algorithm of the balanced binary tree is extended as follows:²

- If the tree is empty, *new* becomes the predecessor and the successor of the *dummy* node (see Figure 7.3b).
- If *new* is inserted in the left branch of a node *r*, the position of *new* in the ordered linked-list is between the predecessor of *r* and *r*. Hence, the predecessor of *r* becomes the predecessor of *new* and *new* becomes the new predecessor of its father *r* (see Figure 7.3c and 7.3e).
- If *new* is inserted in the right branch of a node *r*, the position of *new* in the ordered linked-list is between *r* and the successor of *r*. Hence, *new* becomes the new successor of *r* and the previous successor of *r* becomes the successor of *new* (see Figure 7.3d and 7.3f).

The deletion operation is extended with the same deletion operation of a linked-list. The predecessor of the deleted node becomes the predecessor of its successor and *vice-versa*. Clearly, the additional operations to maintain the ordered linked-list are executed in a constant time. Hence, the overall complexity of the standard operations of a balanced binary tree is not impacted by the additional operations needed to maintain the ordered linked-list.

²Predecessor and successor pointers have to be updated symmetrically.

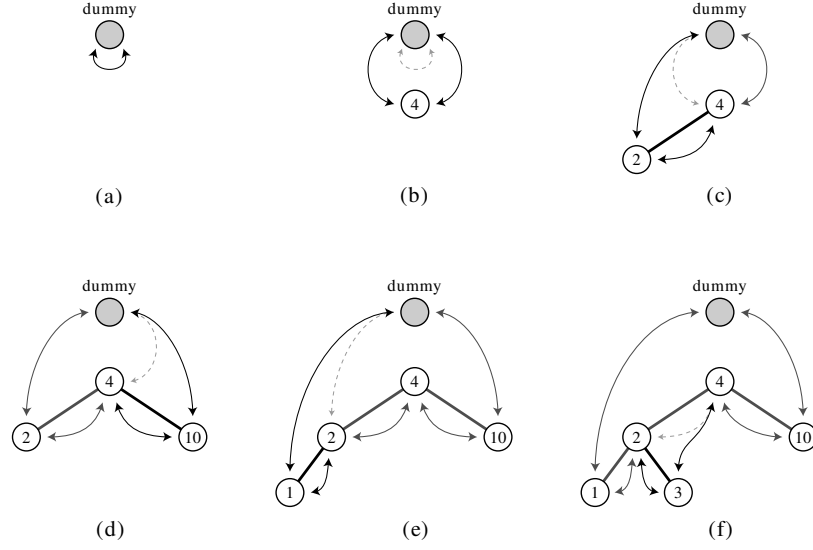


Figure 7.3: Iterative construction of a balanced binary linked-tree. Hatched arrows represents the updated predecessor/successor pointers.

7.2.2 Domination-free insertion

We describe now the algorithm used to extend the insertion operation of a balanced linked-tree into a domination-free insertion operation. Without any loss of generality, we choose to order the solution of the linked-list w.r.t. the first objective. Three cases have to be considered when inserting a candidate solution in the archive.

The tree is empty

The archive is empty. In this case, the insertion operation is exactly the same as the insertion operation of a balanced linked-tree.

Left branch

The candidate solution sol has to be inserted in the left branch of a solution r . Since $sol(obj_1) \leq r(obj_1)$, we have to check that sol does not dominate r . If it is the case, we have to *clean* the archive by iteratively removing r and its successors until one successor is not dominated by sol anymore. This situation is illustrated in Figure 7.4 where the solutions r , r' , and r'' are dominated by sol . The candidate solution is inserted in the tree once all the dominated solutions have been removed.

If r is not dominated by the candidate solution sol , we have to check that sol is not dominated by a solution in the archive. Clearly, if sol is dominated by a solution in the archive, then sol is dominated by the predecessor p of r . We distinguish both following situations:

- The predecessor p of r dominates sol . Then, sol is discarded and does not affect the structure (see Figure 7.5 left).

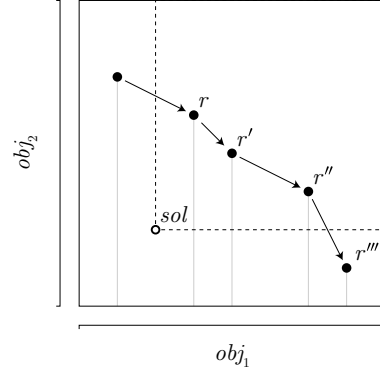


Figure 7.4: The solution r and all its successors until r''' are dominated by the candidate solution sol . Arrows correspond to the successor pointers.

- The predecessor p of r does not dominate sol . In this case, sol does not dominate and is not dominated by some solutions in the archive. The, candidate solution sol is inserted in the tree and respects the domination-free property (see Figure 7.5 right).

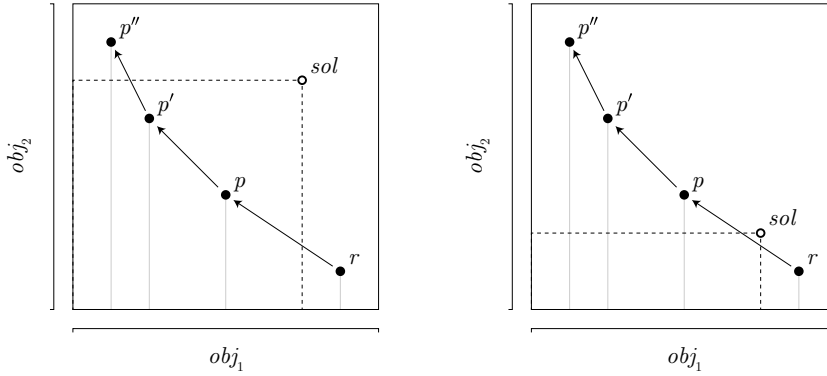


Figure 7.5: If it exists, the predecessor p of r either dominates (left) or does not dominate (right) the candidate solution sol .

Right branch

The candidate solution sol has to be inserted in the right branch of a solution r . Since $sol(obj_1) > r(obj_1)$, we have to check that sol is not dominated by r . If it is the case, the candidate solution sol is discarded and does not affect the archive. If r does not dominate sol , we have to check that the candidate solution does not dominate some solutions in the archive. Clearly, if sol dominates some solutions in the archive, then sol dominates the successor s of r . We distinguish both following possible executions:

- The successor s of r is dominated by sol . Then, the archive is cleaned in the same way as in the left branch insertion (see Figure 7.6 left).

- The successor s of r is not dominated by sol . In this case, sol does not dominate and is not dominated by some solutions in the archive. The candidate solution sol is inserted in the tree and respects the domination-free property (see Figure 7.6 right).

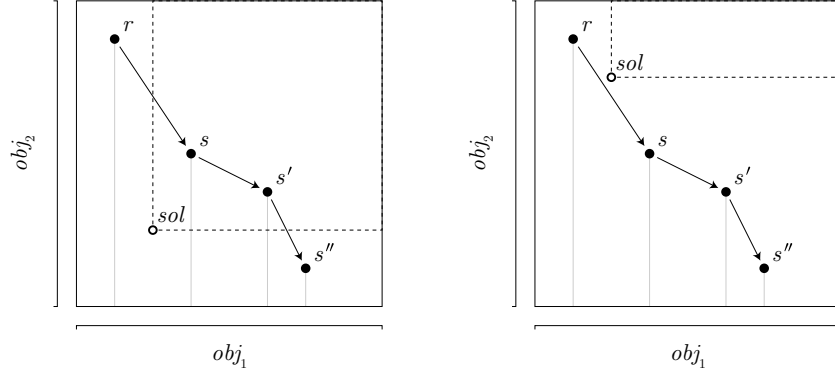


Figure 7.6: If it exists, the successor s of r is either dominated (left) or not dominated (right) the candidate solution sol .

The pseudo-codes of both *left insertion* and *right insertion* are presented in Pseudo-codes 7.1 and 7.2 respectively.

```

1  in  $sol$  : the candidate solution
2  in  $r$  : the possible father of  $sol$  in the tree
3  method INSERTLEFT( $r, sol$ ):
4      if  $r(obj_2) \geq sol(obj_2)$  then
5          CLEAN( $r, sol$ )
6          INSERT( $sol$ )
7      else if HASPRED( $r$ ) then
8           $p \leftarrow \text{PRED}(r)$ 
9          if  $p(obj_2) \leq sol(obj_2)$  then
10             DISCARD( $sol$ )
11         else
12             INSERT( $new$ )
13     else
14         INSERT( $new$ )

```

Pseudo code 7.1

7.2.3 Complexity

The domination-free balanced linked-tree is able to decide if a candidate solution dominates or is dominated by some solutions in the archive with a time complexity of $\Theta(\log n)$, that is the number of operations needed to find the position of the candidate solution in the ordered linked-list. The *clean* operations is the

```

1  in sol : the candidate solution
2  in r : the possible father of sol in the tree
3  method INSERTRIGHT(r, sol):
4      if  $r(obj_2) \leq sol(obj_2)$  then
5          DISCARD(sol)
6      else if HASUCC(r) then
7           $s \leftarrow \text{SUCC}(r)$ 
8          if  $s(obj_2) \geq sol(obj_2)$  then
9              CLEAN(s, sol)
10         INSERT(new)
11      else
12         INSERT(new)

```

Pseudo code 7.2

bottle neck of this data structure. Indeed, removing a solution from a balanced linked-tree has a time complexity of $\mathcal{O}(\log n)$ due to the need of rebalancing the tree. Hence, the *clean* operation as an overall time complexity of $\mathcal{O}(\Delta \log n)$ where Δ is the number of solutions to remove. All in one, the time complexity of the domination-free insertion is between $\Omega(\log n)$ and $\mathcal{O}(n \log n)$.

7.3 Quad-trees

A Quad-tree [16, 23] is a data structure that generalizes binary search trees to store m -dimensional vectors. Any node of a quad-tree divides its subspace into 2^m disjoint subspaces. More precisely, the root of the tree divides the space into 2^m subspaces originating at the root. The 2^m sons of the root (exactly one for each subspace of the root) divides their subspaces into 2^m subspaces and so on (see Figure 7.7).

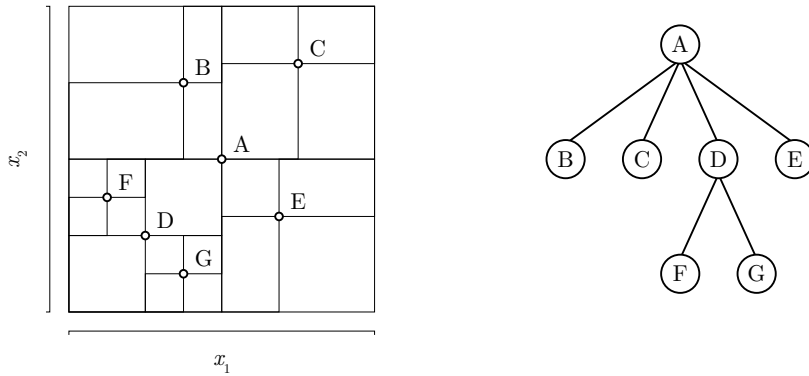


Figure 7.7: Illustration of the space partition (left) of an arbitrary bi-dimensional quad-tree (right). The node A is the root of the quad-tree and the nodes B,C,D, and E are the sons of A. Nodes F and G are the sons of node D.

The insertion of a new element in a quad-tree is analogous to the insertion operation of a binary search tree. The only difference from the binary search tree is the need to perform m comparisons at each node of the quad-tree to determine the son in which the new element has to be inserted. The 2^m different sons of a quad-tree are defined by a successorship function that is defined as follows:

Definition 7.1 (k -successor). A node x is called k -successor of a node y if and only if:

$$k = \sum_{i=1}^m k_i 2^{m-i} \quad \text{with} \quad k_i = \begin{cases} 1 & \text{if } x_i \leq y_i \\ 0 & \text{if } x_i > y_i \end{cases}$$

Definition 7.2 (k -son). We said that x is a k -son of a node y if x is a k -successor of y and y is the father of x .

Figure 7.8 illustrates the successorship function on an arbitrary three-dimensional quad-tree.

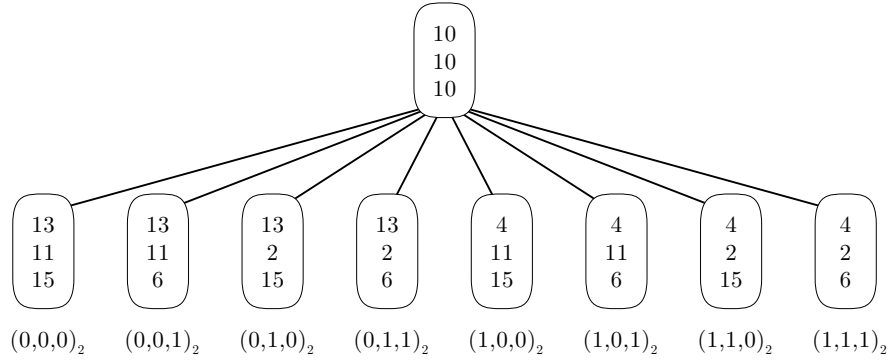


Figure 7.8: Each node of a m -dimensional quad-tree has a maximum of 2^m sons. The k -successors of the root are represented as binary numbers $k = (k_1, \dots, k_m)_2$.

The removal operation of a quad-tree is more complex than its insertion operation and is out of the scope of this master's thesis. The interested reader may refer to [45] for more information on quad-trees and multi-dimensional data structures.

7.3.1 Domination-free Quad-Trees

Quad-trees seem natural for storing multi-dimensional vectors and can be used to store archives. The idea of using quad-trees to store an archive was initially proposed by Habenicht [23] and later specified by Sun and Steuer [52, 53].³ Figure 7.9 illustrates the similarities between an archive and its possible representation in a quad-tree.

Let x be a k -successor of y such that $(k_1, \dots, k_m)_2$ is the binary representation of k . We define the following sets:

$$\begin{aligned} S_1(k) &= \{i \mid k_i = 1\}, \\ S_0(k) &= \{i \mid k_i = 0\}. \end{aligned} \tag{7.1}$$

³In [52], Sun extended the algorithm proposed in [53]. The idea is to reduce the loss of memory due to the empty sons, by using linked-lists instead of arrays. The fundamental idea behind the algorithm however stays the same.

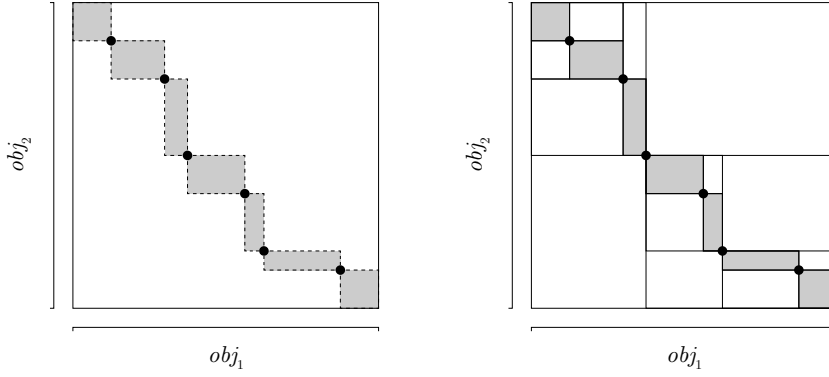


Figure 7.9: An arbitrary bi-objective archive (left) and its possible representation in a quad-tree (right). Gray areas correspond to the diversification space of the archive.

In other words, $S_1(k)$ is the set of objectives for which x is better than y . Conversely, $S_0(k)$ is the set of objectives for which x is worse than y .⁴ According to Habenicht [23], we can use this information to find out dominance relations indirectly. Let x be a k -successor of y and z be a l -successor of y :

$$\begin{aligned} x \prec z & : S_0(k) \subseteq S_0(l), \\ x \prec z & : S_1(l) \subseteq S_1(k). \end{aligned} \quad (7.2)$$

As a consequence of the definition of the successorship function (Definition 7.1), we observe that:

$$\begin{aligned} x \prec y & \text{ if and only if } k = 2^m - 1, \\ y \prec x & \text{ if and only if } \forall i \in S_1(k) : x_i = y_i. \end{aligned} \quad (7.3)$$

Therefore, each node of a domination-free quad-tree has at most $2^m - 2$ sons. Particularly, a m -dimensional domination-free quad-tree becomes a binary tree if $m = 2$ (see Figure 7.9). All in one, we can use the information of Equations 7.1, 7.2 and 7.3 to define the following sets:

$$\begin{aligned} \text{stronger}(k) &= \{l \in \{1, \dots, 2^m - 2\} \mid S_1(k) \subseteq S_1(l)\}, \\ \text{weaker}(k) &= \{l \in \{1, \dots, 2^m - 2\} \mid S_0(k) \subseteq S_0(l)\}. \end{aligned} \quad (7.4)$$

7.3.2 Domination-free insertion

We now have all the definitions needed to present the main components of Habenicht's insertion algorithm [23]. Each time a candidate solution has to be inserted in the quad-tree, we have to ensure that the domination-free property of the quad-tree is respected. Two questions have to be considered:

1. Is the candidate solution dominated by at least one solution in the quad-tree ?
2. Does the quad-tree contain solutions that are dominated by the candidate solution ?

⁴Better and worse are used in a minimization context.

Without going into the details of the insertion algorithm, the remainder of this section presents the general ideas to address both of the previous questions.

The candidate is dominated

The first step of the algorithm is to determine if the candidate solution is dominated by at least one solution already contained in the archive. If it is the case, the candidate solution is discarded and the algorithm stops. In the other case, the second step of the insertion algorithm is executed. Let r be the root of the quad-tree and sol be the candidate solution, we define the method `ISDOMINATED( $r, sol$ )` as follows:

1. If $r \prec sol$, then sol has to be discarded and the insertion algorithm stops.
- 2.1. Else, determine k such that sol is a k -successor of r .
- 2.2. For all l -son v of r such that $l \in stronger(k)$, call recursively the algorithm `ISDOMINATED( $v, sol$ )`.

The candidate dominates

The second step of the insertion algorithm consists in removing all the solutions already contained in the archive that are dominated by the candidate solution. Let r be the root of the quad-tree and sol the candidate solution, we defined the method `CLEAN( $r, sol$ )` as follows:

1. If $sol \prec r$, then r is dominated by sol and has to be *deleted*.
2. Determine k such that sol is a k -successor of r .
3. For all l -son v of r such that $l \in weaker(k)$, call recursively the algorithm `CLEAN( $v, sol$ )`

Deletion

The deletion operation is called each time a solution of the quad-tree has to be removed from the structure. This operation presents some difficulties since the deletion of a node destroys the structure on which the subtrees of that node are based. This situation is illustrated in Figure 7.10 where the root of the quad-tree is the only node that is dominated by the candidate solution. In the implementation of the insertion algorithm proposed by Habenicht [23] and later by Sun and Steuer [53], each time a node is deleted, all the element of its subtrees are reinserted, one by one, in the whole quad-tree. *Of course, this is an unsatisfactory approach, and further investigations have to be done on this matter.* [23]

7.3.3 Improved deletion

This section presents a new algorithm to improve the deletion algorithm of Habenicht [23] and Sun and Steuer [53]. The aim of the new algorithm is to detect subtrees that are entirely dominated by the candidate solution. Hence, one can removes all the nodes of a dominated subtree all at once instead of one by one.

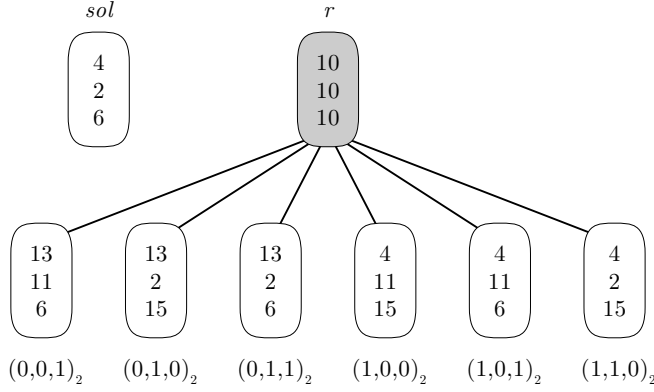


Figure 7.10: The root r of the quad-tree is dominated by the candidate solution sol . However, none of its sons is dominated by the candidate solution. It is thus necessary to repair the structure of the quad-tree after removing its root r .

Before presenting the idea behind the new algorithm, let us introduce the following definitions:

Definition 7.3 (Bound quad-tree). Let $sols(r)$ denote all the solutions contained in the quad-tree rooted at r . We said that r is bound on objective i if and only if:

$$\forall sol' \in sols(r), sol'(obj_i) \geq sol(obj_i).$$

Clearly, all the subtrees of r are also bound on objective i .

Definition 7.4 (Bound vector). Consider a quad-tree rooted at r and a candidate solution sol , the bound vector BV^r of r is defined as follows:

$$BV^r = (BV_1^r, \dots, BV_m^r)$$

$$BV_i^r = \begin{cases} 1 & \text{iff } \forall sol' \in sols(r) : sol'(obj_i) \geq sol(obj_i) \\ 0 & \text{otherwise} \end{cases}$$

Clearly, r is weakly-dominated by sol if and only if $BV^r = (1, \dots, 1)$.

For instance, consider the quad-tree r and the candidate solution sol depicted in Figure 7.11. On the left, the quad-tree r is bound on no objective $BV^r = (0, 0)$. Each k -son of r may contain solutions that are dominated as well as solutions that are not dominated by the candidate solution. The right part of Figure 7.11 presents the same situation with the difference that r is now bound on the first objective $BV^r = (1, 0)$. We observe that the $(1, 0)_2$ -son of r is now bound on all the objectives (*i.e.* it is entirely dominated by the candidate solution) and can thus be removed.

The following definition generalizes the use of bound vectors to detect dominated subtrees of a quad-tree.

Definition 7.5 (Dominated k -sons). Consider a quad-tree rooted at r such that r is dominated by a candidate solution sol . We define the set of the k -sons of r that are entirely dominated by the candidate solution as follows:

$$dominated(BV^r) = \{k \in \{1, \dots, 2^m - 2\} \mid S_1(k) \subseteq S_1(BV^r)\}$$

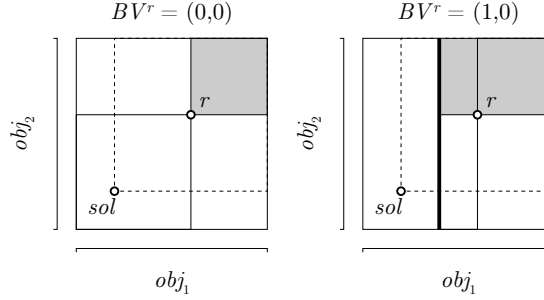


Figure 7.11: A bound vector provides additional information to detect dominated subtrees. The gray areas represent the k -sons of r that are entirely dominated by sol .

Observe that the number of dominated subtrees grows exponentially with the number of 1 in the bound vector.

Computing the bound vector

At this point, all we need to know is how to compute bound vectors. The bound vector of a subtree is built on the branch from the root of the quad-tree to the root of the subtree.

For instance, let sol be a candidate solution and r be the root of a quad-tree such that $r(obj_i) > sol(obj_i)$. As a consequence of the transitivity of the successorship function, all the k -sons s of r such that $k_i = 0$ respect the following property:

$$\forall sol' \in sols(s) : sol'(obj_i) > sol(obj_i).$$

Hence the quad-tree rooted at s is bound on objective i : $BV_i^s = 1$.

More formally, consider r the root of a quad-tree with BV^r its bound vector such that r is a k -successor of sol and s is a l -son of r . The bound vector BV^s of s is defined as follows:

$$BV_i^s = BV_i^r \vee \neg(k_i \vee l_i)$$

Notice that the bound vector of the root of the whole quad-tree is initialized at $(0, \dots, 0)$.

7.3.4 Complexity

The computational complexity of the domination-free insertion presented in this section depends highly from the shape of the quad-tree used to store the archive. The shape of a quad-tree is determined by the order in which solutions are inserted into it. Algorithms have been proposed to reduce the size of a quad-tree [16, 45], but do not ensure that the quad-tree is balanced.

As a consequence of Equation 7.4, we observe that no more than $\frac{2^m-2}{2}$ nodes have to be considered at each step of the ISDOMINATED procedure. Hence, supposing that the quad-tree is balanced, the worst case complexity of this operation is defined by the following formula:

$$\sum_{i=0}^{\log_a n} \left(\frac{a}{2}\right)^i \quad \text{with} \quad a = 2^m - 2$$

The new algorithm proposed for deletion allows to detect subtrees that are entirely dominated by the candidate solution. This algorithm has not yet been the object of a performance assessment but it seems likely that it can bring a substantial overall speed-up to the `CLEAN` procedure.

Conclusion

This master’s thesis presents the Multi-Objective Large Neighborhood Search framework: a new generic way of using constraint programming to solve multi-objective combinatorial optimization problems. MO-LNS is an attempt to answer the lack of efficient constraint programming methods in the field of multi-objective optimization [15].

Among our contributions, we have introduced a new global constraint called the **Pareto** constraint. The aim of the **Pareto** constraint is to prune branches of the search tree that are dominated by the already discovered solutions. The **Pareto** constraint improves the work of Gavanelli [19] by ensuring idempotent filtering. Clearly, the **Pareto** constraint is a core component of MO-LNS to ensure the discovery of solutions that strictly improve the quality of the archive.

During our experiments, we have observed that the selection of the next solution to optimize is one of the critical decisions to be taken by MO-LNS. Indeed, it is important to make sure that the neighborhood of this solution contributes to the improvement of the archive and prevents MO-LNS from useless restarts. With this in mind, we have proposed and compared different selection strategies that could be used to select the next solution to optimize.

From a performance point of view, we have shown that MO-LNS outperforms current state of the art MOCO methods based on constraint programming. We have demonstrated the flexibility of MO-LNS by providing efficient solutions on standard academic problems as well as on a complex industrial problem. MO-LNS seems well suited for complex problems involving many side constraints. This is particularly interesting since this kind of problems cannot be tackled easily with current state of the art EA/LS methods.

The reusability of our work is ensured through the modeling abstractions that we have developed in the open source OscaR solver [38]. In that way, Chapter 5 provides the reader with a first introduction on how to use MO-LNS and MO-CP to tackle multi-objective problems in a clean and easy way. To the best of our knowledge, these functionalities appear in no other CP solver.

Finally, we have presented three different data structures to implement an archive. The domination-free balanced linked-tree that we have developed seems to be a good candidate for storing bi-objective archive. Besides, we have presented a new deletion algorithm to improve the well known domination-free quad-trees [23, 53].

Future Work

MO-LNS is still in its infancy and many aspects of the framework have to be studied. The following presents some directions that could be explored.

Complex problems. As observed, MO-LNS seems particularly well suited for complex problems in which many side constraints are involved. We would like to measure the performance of MO-LNS on complex problems as scheduling problems and routing problems.

Selection strategies. Selecting the next solution to optimize is one of the most critical responsibilities of MO-LNS. In this master's thesis, we have presented some selection strategies to select the solution that is the most likely to improve the quality of the archive. These strategies are mainly based on the objective space. The development of new selection strategies based on the decision space have to be considered.

MO-Relaxation. Single-objective relaxation may be too restrictive for some problems. Hence, the development of generic multi-objective relaxation has to be investigated. Particularly, concepts of the field of population based method may contribute a lot to the performance of MO-LNS. For instance, *path-relinking* [18] seems a promising idea for the implementation of multi-objective relaxations.

Parallelization. MO-LNS presents a natural tendency to be parallelized. A first idea could be to implement the archive as a parallel data structure in which parallel agents could pick solutions and perform whole MO-LNS iterations in their respective threads.

Data structure. Apart from MO-LNS, we have presented some data structures to implement archives. The new deletion algorithm that we have proposed in Section 7.3.3 has to be compared against standard implementations of domination-free quad-trees [23, 53].

Bibliography

- [1] Ajith Abraham and Lakhmi Jain. Evolutionary multiobjective optimization. *Evolutionary Multiobjective Optimization*, 2005.
- [2] G. Adelson-Velskii and E. Landis. An algorithm for the organization of information. Technical report, DTIC Document, 1963.
- [3] Vincent Barichard and Jin-Kao Hao. A population and interval constraint propagation algorithm for multi-objective optimization. In *Metaheuristics International Conference-MIC*, volume 3. Citeseer, 2003.
- [4] Rudolf Bayer. Symmetric binary b-trees: Data structure and maintenance algorithms. *Acta informatica*, 1(4):290–306, 1972.
- [5] Nicolas Beldiceanu, Mats Carlsson, Sophie Demasse, and Thierry Petit. Global constraint catalogue: Past, present and future. *Constraints*, 12(1):21–62, 2007.
- [6] R. Bent and P.V. Hentenryck. A two-stage hybrid algorithm for pickup and delivery vehicle routing problems with time windows. *Computers & Operations Research*, 33(4):875–893, 2006.
- [7] Christian Bessière and Jean-Charles Régin. Mac and combined heuristics: Two reasons to forsake fc (and cbj) on hard problems. In *Principles and Practice of Constraint Programming: CP96*, pages 61–75. Springer, 1996.
- [8] Christian Bessière and Pascal Van Hentenryck. To be or not to be... a global constraint. In *Principles and Practice of Constraint Programming-CP 2003*, pages 789–794. Springer, 2003.
- [9] Frédéric Boussemart, Fred Hemery, Christophe Lecoutre, and Lakhdar Sais. Boosting systematic search by weighting constraints. In *ECAI*, volume 16, page 146, 2004.
- [10] S. Cahon, N. Melab, and E.-G. Talbi. Paradiseo: A framework for the reusable design of parallel and distributed metaheuristics. *Journal of Heuristics*, 10(3):357–380, May 2004.
- [11] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-2. *Evolutionary Computation, IEEE Transactions on*, 6(2):182–197, 2002.
- [12] Yves Deville and Pascal Van Hentenryck. Course notes: Constraint Programming (INGI2365). Université Catholique de Louvain, 2012.

- [13] Juan J. Durillo and Antonio J. Nebro. jmetal: A java framework for multi-objective optimization. *Advances in Engineering Software*, 42:760–771, 2011.
- [14] Matthias Ehrgott. *Multicriteria optimization*, volume 2. Springer Berlin, 2005.
- [15] Matthias Ehrgott and Xavier Gandibleux. Hybrid metaheuristics for multi-objective combinatorial optimization. *Hybrid metaheuristics*, pages 221–259, 2008.
- [16] Raphael A. Finkel and Jon Louis Bentley. Quad trees a data structure for retrieval on composite keys. *Acta informatica*, 4(1):1–9, 1974.
- [17] Eugene C Freuder. In pursuit of the holy grail. *Constraints*, 2(1):57–61, 1997.
- [18] Xavier Gandibleux, Hiroyuki Morita, and Naoki Katoh. Impact of clusters, path-relinking and mutation operators on the heuristic using a genetic heritage for solving assignment problems with two objectives. In *Proceedings of The Fifth Metaheuristics International Conference MIC’03*, 2003.
- [19] M. Gavanelli. An algorithm for multi-criteria optimization in csps. *ECAI*, 2:136–140, 2002.
- [20] Fred Glover. Tabu search: A tutorial. *Interfaces*, 20(4):74–94, 1990.
- [21] Solomon W Golomb and Leonard D Baumert. Backtrack programming. *Journal of the ACM (JACM)*, 12(4):516–524, 1965.
- [22] Inc. Gurobi Optimization. Gurobi optimizer reference manual, 2013.
- [23] W Habenicht. Quad trees, a datastructure for discrete vector optimization problems. In *Essays and Surveys on Multiple Criteria Decision Making*, pages 136–145. Springer, 1983.
- [24] Yacov Y Haimes, Leon S Lasdon, and David A Wismer. On a bicriterion formulation of the problems of integrated system identification and system optimization. *IEEE Transactions on Systems, Man, and Cybernetics*, 1(3):296–297, 1971.
- [25] Michael Pilegaard Hansen and Andrzej Jaszkievicz. *Evaluating the quality of approximations to the non-dominated set*. IMM, Department of Mathematical Modelling, Technical University of Denmark, 1998.
- [26] Robert M Haralick and Gordon L Elliott. Increasing tree search efficiency for constraint satisfaction problems. *Artificial intelligence*, 14(3):263–313, 1980.
- [27] William D Harvey. *Nonsystematic backtracking search*. PhD thesis, Cite-seer, 1995.
- [28] William D Harvey and Matthew L Ginsberg. Limited discrepancy search. In *International Joint Conference on Artificial Intelligence*, volume 14, pages 607–615. LAWRENCE ERLBAUM ASSOCIATES LTD, 1995.

- [29] Joshua Knowles. Towards landscape analyses to inform the design of a hybrid local search for the multiobjective quadratic assignment problem. *Soft computing systems: design, management and applications*, 2002:271–279, 2002.
- [30] Joshua Knowles and David Corne. Instance generators and test suites for the multiobjective quadratic assignment problem. In *Second International Conference on Evolutionary Multi-Criterion Optimization EMO 2003*, number 2632 in LNCS, pages 295–310. Springer, 2003. Available from <http://www.cs.man.ac.uk/~jknowles/mQAP/>.
- [31] P. Laborie and D. Godard. Self-adapting large neighborhood search: Application to single-mode scheduling problems. *Proceedings MISTA-07, Paris*, pages 276–284, 2007.
- [32] Philippe Laborie and Daniel Godard. Self-adapting large neighborhood search: Application to single-mode scheduling problems. *Proceedings MISTA-07, Paris*, pages 276–284, 2007.
- [33] Thibaut Lust and Jacques Teghem. The multiobjective multidimensional knapsack problem: a survey and a new approach. *International Transactions in Operational Research*, 19(4):495–520, 2012.
- [34] Alan K Mackworth. *On reading sketch maps*. Department of Computer Science, University of British Columbia, 1977.
- [35] D. Mehta, B. O’Sullivan, and H. Simonis. Comparing solution methods for the machine reassignment problem. In *Principles and Practice of Constraint Programming*, pages 782–797. Springer, 2012.
- [36] Herve Meunier, E-G Talbi, and Philippe Reininger. A multiobjective genetic algorithm for radio network optimization. In *Evolutionary Computation, 2000. Proceedings of the 2000 Congress on*, volume 1, pages 317–324. IEEE, 2000.
- [37] Martin Odersky, Lex Spoon, and Bill Venners. *Programming in Scala*. Artima Inc, 2008.
- [38] OscaR Team. OscaR: Scala in OR, 2012. Available from <https://bitbucket.org/oscarlib/oscar>.
- [39] D. Pacino and P. Van Hentenryck. Large neighborhood search and adaptive randomized decompositions for flexible jobshop scheduling. In *Proceedings of the Twenty-Second international joint conference on Artificial Intelligence-Volume Volume Three*, pages 1997–2002. AAAI Press, 2011.
- [40] V Pareto and Manuale di Economia Politica. Societa editrice libraria, milano, italy, 1906. translated into english by as schwier as manual of political economy, 1971.
- [41] J.C. Régim. Global constraints and filtering algorithms. *Constraint and Integer Programming-Towards a Unified Methodology*, pages 89–129, 2003.

- [42] Jean-Charles Régin. A filtering algorithm for constraints of difference in cps. In *Proceedings of the national conference on artificial intelligence*, pages 362–362. JOHN WILEY & SONS LTD, 1994.
- [43] Stephen V Rice. Braided avl trees for efficient event sets and ranked sets in the SIMSCRIPT III simulation programming language. In *Proceedings of the 2007 Western Multiconference on Computer Simulation: International Conference on High Level Simulation Languages and Applications*, pages 150–155, 2007.
- [44] Francesca Rossi, Peter Van Beek, and Toby Walsh. *Handbook of constraint programming*. Elsevier Science, 2006.
- [45] Hanan Samet. *Foundations of multidimensional and metric data structures*. Morgan Kaufmann, 2006.
- [46] P. Schaus, P. Van Hentenryck, J.N. Monette, C. Coffrin, L. Michel, and Y. Deville. Solving steel mill slab problems with constraint-based techniques: Cp, lns, and cbls. *Constraints*, 16(2):125–147, 2011.
- [47] Pierre Schaus. Variable objective large neighborhood search. *Submitted to CP13*, 2013.
- [48] Pierre Schaus, Jean-Charles Régin, Rowan Van Schaeren, Wout Dullaert, and Birger Raa. Cardinality reasoning for bin-packing constraint: application to a tank allocation problem. In *Principles and Practice of Constraint Programming*, pages 815–822. Springer, 2012.
- [49] Meinolf Sellmann. Approximated consistency for knapsack constraints. In *Principles and Practice of Constraint Programming—CP 2003*, pages 679–693. Springer, 2003.
- [50] P. Shaw. Using constraint programming and local search methods to solve vehicle routing problems. *Principles and Practice of Constraint Programming—CP98*, pages 417–431, 1998.
- [51] Christine Solnon. *Ant colony optimization and constraint programming*. Wiley-ISTE, 2010.
- [52] Minghe Sun. A primogenitary linked quad tree data structure and its application to discrete multiple criteria optimization. *Annals of Operations Research*, 147(1):87–107, 2006.
- [53] Minghe Sun and Ralph E Steuer. Quad-trees and linear lists for identifying nondominated criterion vectors. *INFORMS Journal on Computing*, 8(4):367–375, 1996.
- [54] El-Ghazali Talbi. *Metaheuristics: from design to implementation*, volume 74. Wiley, 2009.
- [55] EL Ulungu and J Teghem. Multi-objective combinatorial optimization problems: A survey. *Journal of Multi-Criteria Decision Analysis*, 3(2):83–104, 1994.

- [56] Pascal Van Hentenryck and Laurent Michel. Nondeterministic control for hybrid search. *Constraints*, 11(4):353–373, 2006.
- [57] Pascal Van Hentenryck, Vijay Saraswat, and Yves Deville. Design, implementation, and evaluation of the constraint language cc (fd). *The Journal of Logic Programming*, 37(1-3):139–164, 1998.
- [58] David L Waltz. Generating semantic description from drawings of scenes with shadows. 1972.
- [59] Luc N. Van Wassenhove and Ludo F. Gelders. Solving a bicriterion scheduling problem. *European Journal of Operations Research*, 4:42–48, 1980.
- [60] Lyndon While, Phil Hingston, Luigi Barone, and Simon Huband. A faster algorithm for calculating hypervolume. *Evolutionary Computation, IEEE Transactions on*, 10(1):29–38, 2006.
- [61] Xavier Gandibleux. A collection of test instances for multiobjective combinatorial optimization problems, 2013. Available from <http://xgandibleux.free.fr/MOCOLib/>.
- [62] Eckart Zitzler, Kalyanmoy Deb, and Lothar Thiele. Comparison of multiobjective evolutionary algorithms: Empirical results. *Evolutionary computation*, 8(2):173–195, 2000.
- [63] Eckart Zitzler, Marco Laumanns, Lothar Thiele, Eckart Zitzler, Eckart Zitzler, Lothar Thiele, and Lothar Thiele. Spea2: Improving the strength pareto evolutionary algorithm, 2001.
- [64] Eckart Zitzler, Lothar Thiele, Marco Laumanns, Carlos M Fonseca, and V Grunert da Fonseca. Performance assessment of multiobjective optimizers: An analysis and review. *Evolutionary Computation, IEEE Transactions on*, 7(2):117–132, 2003.